AventuraTrips: Desarrollo de una Aplicación Web para la Gestión de Paquetes Turísticos

Mateo Tascón Gómez y Janire Longarte González

Centro de Formación Prometeo by ThePower

2.º DAW · Desarrollo de Aplicaciones Web (Modalidad Online)

Tutor: Raúl Albiol

3 de mayo de 2026

Resumen

El presente proyecto consiste en el desarrollo de una aplicación web para una agencia mayorista especializada en la organización de paquetes turísticos, denominada AventuraTrips. El objetivo principal es crear una plataforma *online* con un alto grado de realismo y funcionalidad, emulando un entorno comercial de e-commerce en el sector de viajes.

La funcionalidad esencial de la web incluye la gestión de usuarios, permitiendo a los clientes crear cuentas y registrarse, así como explorar nuestro catálogo completo de viajes. Los usuarios pueden interactuar con los productos añadiéndolos a una lista de favoritos y, proceder a la compra de los paquetes turísticos.

En cuanto a la arquitectura técnica, el *frontend* ha sido desarrollado utilizando la librería React (Meta Platforms, 2024), lo que garantiza una interfaz de usuario dinámica y moderna. Para el *backend*, hemos empleado el *framework* Java con Spring Boot (Broadcom, 2024), Spring Security . La persistencia de los datos se gestiona mediante una base de datos MySQL.

La implementación de este proyecto nos ha permitido adquirir y consolidar conocimientos avanzados en el desarrollo de aplicaciones web full-stack, incluyendo el manejo de peticiones API REST, la gestión de estados complejos con React, la autenticación y autorización de usuarios, el diseño de bases de datos relacionales, y la integración de *frameworks* robustos tanto en el lado del cliente como en el del servidor.

Finalmente, es importante destacar que todo el material gráfico, incluidas las imágenes de los destinos utilizadas en la web, son completamente originales, habiendo sido creadas por nosotros mismos utilizando herramientas de Inteligencia Artificial generativa.

Además, el proyecto cuenta con un chatbot diseño por nosotros capaz de responder dudas sencillas.

Abstract Inglés

This project consists of the development of a web application for a wholesale agency specialized in organizing tour packages, called AventuraTrips. The main objective is to create an online platform with a high level of realism and functionality, simulating a commercial e-commerce environment in the travel sector.

The core functionality of the website includes user management, allowing customers to create accounts and register, as well as browse our complete travel catalog. Users can interact with the products by adding them to a favorites list and proceed to purchase the tour packages.

Regarding the technical architecture, the frontend has been developed using the React library, which ensures a dynamic and modern user interface. For the backend, we used Java with the Spring Boot framework. Data persistence is managed through a MySQL database.

The implementation of this project has allowed us to acquire and consolidate advanced knowledge in full-stack web application development, including handling REST

API requests, managing complex states with React, user authentication and authorization, relational database design, and the integration of robust frameworks on both the client and server sides.

Finally, it is important to highlight that all the graphic material, including the destination images used on the website, is completely original and was created by us using generative Artificial Intelligence tools.

In addition, the project features a chatbot designed by us that can answer simple questions.

Índice

Resumen....2

Índice....4

Justificación del Proyecto....7

Introducción....8

Principales requisitos del proyecto....9

Objetivos del Proyecto....10

Objetivo General....10

Objetivos Específicos....11

Descripción....19

Arquitectura de la Solución....19

Diagrama Lógico de Flujo....20

Explicación del Diagrama....22

Diseño de la Base de Datos 22
Diagrama Entidad-Relación 23
Descripción de las Tablas 25
Relaciones y Cardinalidades 29
Estrategia de Claves Primarias e Indexación 31
Tipos de Datos y Restricciones 32
Diseño de Interfaces 33
Tecnologías y Herramientas 43
Tecnologías Frontend 43
Tecnologías Backend 44
Gestión de Datos y Almacenamiento 46
Herramientas de Desarrollo (Tooling) 48
Metodología 48
Metodología empleada y justificación 48
Planificación inicial vs real 49
Análisis de desviaciones 50
Diagrama de Gantt 51
Presupuesto 52
README y control de versiones (GIT) 53
Desarrollo del Backend 54
Estructura del Proyecto 54

Seguridad y Autenticación 56

Flujo de Recuperación de Contraseña 59

API REST: Endpoints y Controladores 60

Proceso de Checkout 62

Manejo Global de Errores 64

Integraciones con Servicios Externos 65

Despliegue con Docker 67

Documentación de la API con Swagger 68

Trabajos futuros 68

Conclusiones 70

Referencias 73

Justificación del Proyecto

La motivación principal que impulsa la creación de este proyecto surge del creciente auge del turismo digital y de la necesidad de contar con plataformas web modernas que faciliten la planificación y compra de experiencias de viaje de forma sencilla, visual e intuitiva. En la actualidad, los usuarios buscan cada vez más gestionar sus viajes de manera autónoma a través de internet, comparando destinos, consultando información detallada y realizando reservas de forma rápida y segura. En este contexto, el desarrollo de AventuraTrips pretende simular un entorno real de comercio electrónico dentro del sector turístico, integrando funcionalidades habituales en este tipo de plataformas y aplicando tecnologías actuales de desarrollo web.

Para la idea nos hemos basado en diversas plataformas consolidadas en el sector de los viajes y experiencias turísticas. Entre ellas destacan Civitatis y WeRoad, que han servido como referencia para el planteamiento y diseño conceptual de este proyecto.

Por un lado, Civitatis se centra principalmente en ofrecer excursiones, visitas guiadas y actividades en múltiples destinos turísticos a nivel internacional. Su modelo se basa en un catálogo amplio de experiencias que los usuarios pueden reservar de forma sencilla a través de su plataforma. La interfaz está diseñada para facilitar la exploración de actividades en diferentes ciudades, priorizando la accesibilidad y la variedad de opciones.

Por otro lado, WeRoad presenta un enfoque algo diferente, ya que está orientado a viajes organizados en grupo, especialmente dirigidos a personas jóvenes que desean compartir experiencias con otros viajeros. Su propuesta destaca por la creación de comunidades de viaje y por una fuerte identidad de marca dirigida a un público dinámico y aventurero.

A partir de estas referencias, AventuraTrips busca integrar elementos de ambos modelos. Por un lado, adopta la idea de un catálogo de viajes accesible y visual, similar al de Civitatis, y por otro incorpora el enfoque de experiencias organizadas y orientadas a un perfil concreto de viajero, inspirado en plataformas como WeRoad.

El público objetivo al que se dirige la aplicación está compuesto principalmente por jóvenes adultos con poder adquisitivo, interesados en vivir experiencias de viaje diferentes, organizadas y de calidad. Se trata de usuarios acostumbrados a utilizar plataformas digitales, que valoran la comodidad de gestionar todo el proceso de búsqueda y compra de viajes desde una única aplicación web.

En conclusión, este proyecto se justifica tanto por su valor educativo en el ámbito del desarrollo de software como por su contextualización dentro de un sector en pleno crecimiento. La creación de AventuraTrips permite explorar y recrear el funcionamiento de plataformas reales del sector turístico, analizando sus características y adaptándolas a un entorno de desarrollo propio.

Introducción

AventuraTrips es una aplicación web diseñada para la gestión y venta de paquetes turísticos a través de una plataforma online. Su objetivo es ofrecer a los usuarios un espacio donde puedan explorar distintos destinos, consultar información detallada de los viajes y realizar su compra de forma rápida y sencilla.

La aplicación centraliza en un único sitio el proceso de búsqueda, selección y compra de experiencias de viaje, mejorando la comodidad y la experiencia del usuario mediante una interfaz moderna y fácil de utilizar.

Entre sus principales funcionalidades se incluyen el registro y gestión de usuarios, la visualización de un catálogo de viajes, la posibilidad de guardar destinos en favoritos y la compra de paquetes turísticos. Además, el sistema se apoya en una arquitectura full-stack basada en React, Java con Spring Boot y MySQL, garantizando una estructura sólida y escalable.

El código fuente completo del proyecto se encuentra disponible públicamente en el repositorio de GitHub https://github.com/Jlongarte/AventuraT (Tascón Gómez & Longarte González, 2026).

Principales requisitos del proyecto

Para garantizar el correcto funcionamiento de la aplicación, el sistema debe cumplir con una serie de requisitos fundamentales:

- Desarrollar una interfaz web moderna y dinámica utilizando React.
- Implementar un backend robusto basado en Java con Spring Boot.
- Diseñar y gestionar una base de datos MySQL para almacenar la información del sistema.
- Permitir la gestión completa de usuarios, incluyendo registro, autenticación y acceso a funcionalidades personalizadas.
- Ofrecer un catálogo organizado de paquetes turísticos accesible para los usuarios.

- Garantizar la comunicación mediante API REST entre frontend y backend.

- Implementar funcionalidades de favoritos y compra de productos.

- Mantener una arquitectura escalable y estructurada que facilite el mantenimiento y futuras ampliaciones del sistema.

En conjunto, estos requisitos permiten construir una aplicación web funcional que reproduce las características principales de una plataforma de comercio electrónico especializada en el sector turístico.

Objetivos del Proyecto

Objetivo General

El objetivo general de este proyecto es diseñar y desarrollar una aplicación web completa de tipo e-commerce orientada al sector turístico, denominada AventuraTrips, que permita a los usuarios explorar, seleccionar y adquirir paquetes turísticos de forma intuitiva, segura y eficiente. La plataforma busca replicar con alto grado de realismo las funcionalidades y experiencias de usuario presentes en plataformas comerciales consolidadas del sector, aplicando para ello tecnologías modernas de desarrollo web tanto en el lado del cliente como en el del servidor.

Objetivos Específicos

Objetivos técnicos del Frontend:

• Desarrollar una interfaz de usuario reactiva y moderna utilizando la librería React, estructurada en componentes reutilizables que faciliten el mantenimiento y la escalabilidad del código.

• Implementar un sistema de enrutamiento dinámico mediante React Router que permita la navegación fluida entre las distintas vistas de la aplicación sin necesidad de recargar la página completa.

• Integrar la comunicación asíncrona con el backend a través de peticiones HTTP utilizando la API Fetch, gestionando de manera eficiente los estados de carga, éxito y error en cada operación.

• Garantizar una experiencia de usuario responsive, adaptando la interfaz a múltiples dispositivos (escritorio, tableta y móvil) mediante el uso de técnicas de diseño adaptativo.

- Optimizar la percepción de carga mediante el diseño e implementación de un componente Loader personalizado, mejorando la retención del usuario durante las peticiones asíncronas.
- Implementar un asistente virtual (Chatbot) integrado en la interfaz para la resolución de dudas sencillas de forma inmediata.

Objetivos técnicos del Backend:

• Construir una API REST robusta y segura utilizando el framework Spring Boot con Java 21, siguiendo una arquitectura de capas (Controller, Service, Repository) que garantice la separación de responsabilidades y la mantenibilidad del código.

• Implementar un sistema completo de autenticación y autorización basado en JSON Web Tokens (JWT) con el algoritmo HS512, integrado con Spring Security, que permita gestionar diferentes roles de usuario (Customer y Company) y proteger los endpoints sensibles de la API.

• Diseñar y administrar una base de datos relacional MySQL con un esquema normalizado que modele correctamente las entidades del dominio (usuarios, clientes, empresas, viajes, guías, reservas, carrito, favoritos e imágenes) y sus relaciones.

• Desarrollar la lógica de negocio necesaria para gestionar el ciclo completo de compra, incluyendo la validación de stock mediante bloqueo pesimista (pessimistic locking), la prevención de reservas duplicadas, el cálculo de precios con soporte multidivisa y el envío de correos electrónicos transaccionales de confirmación.

• Integrar servicios externos de terceros como Cloudinary para la gestión de imágenes en la nube y Resend para el envío de correos electrónicos transaccionales, garantizando una arquitectura desacoplada y extensible.

Objetivos funcionales:

• Permitir el registro y autenticación de dos tipos de usuarios diferenciados: clientes (Customer) que pueden explorar, guardar en favoritos y comprar viajes, y empresas (Company) que pueden crear y gestionar guías turísticos y paquetes de viaje.

• Implementar un catálogo de viajes con paginación, filtrado por descuentos y visualización detallada de cada paquete turístico, incluyendo galería de imágenes, descripción, fechas, precios y disponibilidad.

• Desarrollar las funcionalidades de carrito de compras y lista de favoritos que permitan a los clientes gestionar sus selecciones antes de proceder al checkout.

• Implementar un flujo completo de recuperación de contraseña mediante código de verificación enviado por correo electrónico, con límite de intentos y expiración temporal por seguridad.

• Desarrollar un sistema de checkout que recoja los datos de facturación del cliente, valide la disponibilidad de plazas, procese las reservas de forma transaccional y envíe un recibo de compra por correo electrónico.

• Implementar un formulario de contacto público que permita a cualquier visitante enviar consultas a la empresa.

Objetivos de despliegue:

• Contenerizar la aplicación backend y la base de datos mediante Docker y Docker Compose, facilitando la portabilidad y el despliegue reproducible del sistema en diferentes entornos.

• Documentar la API REST de manera interactiva mediante Swagger/OpenAPI, proporcionando una referencia completa y accesible para desarrolladores y evaluadores.

	R - Requisito	F - Función	T - Tarea	P - Prueba
R01	Solo los usuarios registrados pueden comprar y guardar favoritos.	R01F01: Gestión de autenticación y autorización (Spring Security + JWT).	T01: Configurar seguridad en el Backend y crear tabla users. T02: Crear formularios de Login/Registro en React.	P01: Registrar un usuario y verificar en MySQL. P02: Intentar acceder a la compra sin login (debe denegar).
R02	El cliente debe poder ver todos los viajes disponibles.	R02F01: Exposición de catálogo mediante API REST.	T01: Crear el modelo Trip y el Repository en Spring Boot.	P01: Hacer un GET api/trips y recibir JSON.

			T02: Desarrollar componente de "Cards" y Grid en React.	P02: Comprobar que los viajes cargan correctamente.
R03	El usuario puede marcar viajes que le gusten para verlos luego.	R03F01: Gestión de persistencia de favoritos.	T01: Crear tabla intermedia en MySQL. T02: Crear endpoint POST/DELETE para gestionar la lista.	P01: Click en "corazón" y verificar que se mantiene al recargar la sesión.
R04	El sistema debe permitir la compra de paquetes turísticos.	R04F01: Procesamiento de pedidos.	T01: Crear lógica de negocio para la entidad.	P01: Finalizar compra y recibir mensaje en el front.

			T02: Crear pasarela de checkout simulada en el Frontend.	
R05	Los usuarios registrados pueden visualizar su historial de viajes comprados.	R05F01: Visualización del Historial.	T01:Integración de API y Consumo de Datos. Consumo del endpoint GET /api/customer/getBookings utilizando el JWT extraído del AuthContext para garantizar seguridad. T02: Diseñar vista de "Mis Viajes" en el Dashboard del usuario.	P01: Seguridad y Privacidad de Datos. Validación de seguridad en el servidor: los viajes se filtran por el ID de usuario contenido en el token, impidiendo el acceso a compras de terceros. Implementación de Rutas Protegidas: Redirección automática al /login si un usuario no

			Implementación de Navegación Contextual: Reutilización del componente CardProduct.jsx en modo "Read-Only" mediante el paso de estados de ruta (location.state), bloqueando botones de compra para viajes ya adquiridos.	autenticado intenta acceder al historial.
R06	El sistema debe asistir al usuario en dudas recurrentes.	R06F01: Chatbot interactivo en el frontend.	T01: Crear lógica de respuestas y diseño de la burbuja de chat.	P01: Interactuar con el chatbot y recibir respuesta coherente.

R07	Mejora de la experiencia de usuario en tiempos de espera.	R07F01: Loader animado personalizado.	T01: Implementar HOC (Higher Order Component) o estado de carga en React.	

Descripción

Arquitectura de la Solución

Para AventuraTrips hemos optado por una Arquitectura de Referencia de Tres Capas con un modelo de desacoplamiento total entre el cliente y el servidor.

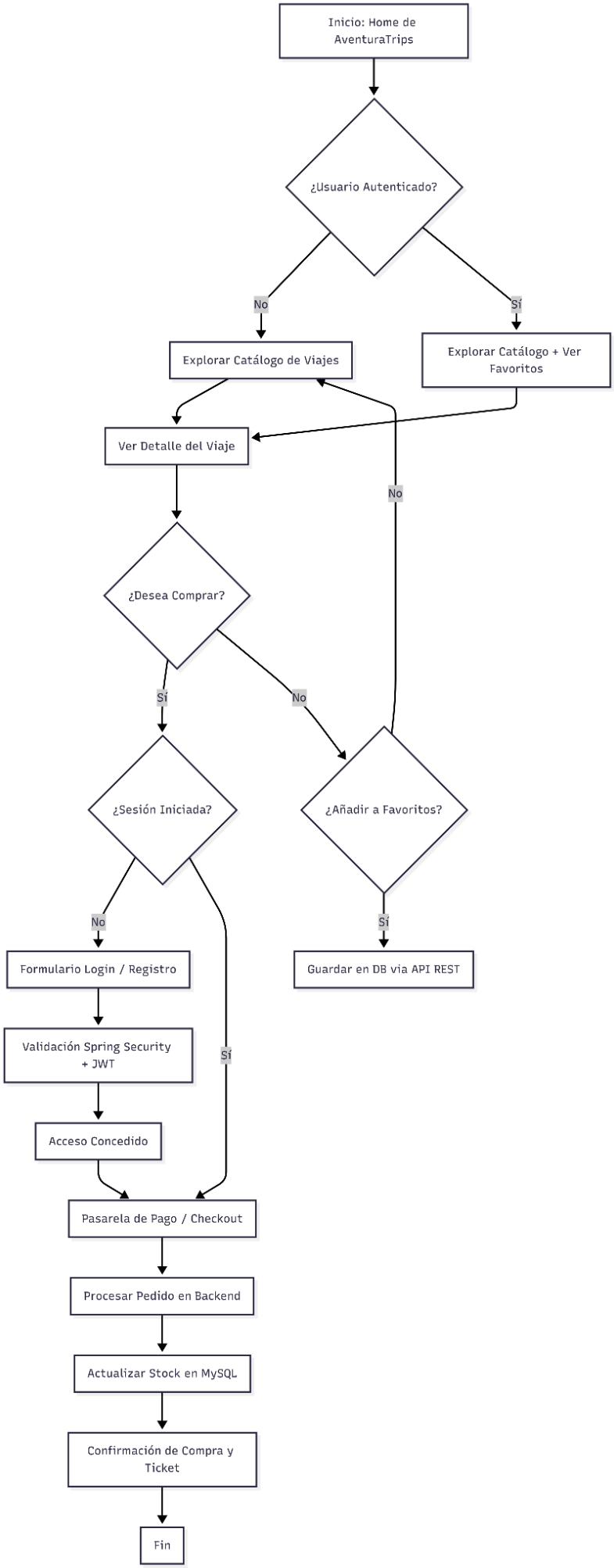
1. Capa de Presentación (Frontend): Desarrollada con React. Se encarga de la gestión del estado de la interfaz, el enrutamiento (React Router) y la comunicación asíncrona con el servidor mediante Fetch.
2. Capa de Negocio (Backend): Basada en Spring Boot.
    - Controller: Define los endpoints de la API REST.
    - Service: Contiene la lógica de negocio (validaciones, cálculos).
    - Security: Filtros de Spring Security para proteger las rutas privadas.
3. Capa de Datos (Persistencia): Una base de datos relacional MySQL.

2.2 El rol de la IA Generativa en el Proyecto

A diferencia de proyectos convencionales, AventuraTrips integra un pipeline de diseño propio. Las imágenes de los destinos no son recursos de stock, sino activos generados mediante IA Generativa. Esto garantiza:

- Originalidad: Estética visual coherente en todo el sitio.
- Derechos: Propiedad intelectual total sobre el material gráfico del proyecto.

Diagrama Lógico de Flujo


Inicio: Home de AventuraTrips
¿Usuario Autenticado?
No
Sí
Explorar Catálogo de Viajes
Explorar Catálogo + Ver Favoritos
Ver Detalle del Viaje
No
¿Desea Comprar?
Sí
No
¿Sesión Iniciada?
¿Añadir a Favoritos?
No
Sí
Formulario Login / Registro
Guardar en DB via API REST
Validación Spring Security + JWT
Sí
Acceso Concedido
Pasarela de Pago / Checkout
Procesar Pedido en Backend
Actualizar Stock en MySQL
Confirmación de Compra y Ticket
Fin

Explicación del Diagrama

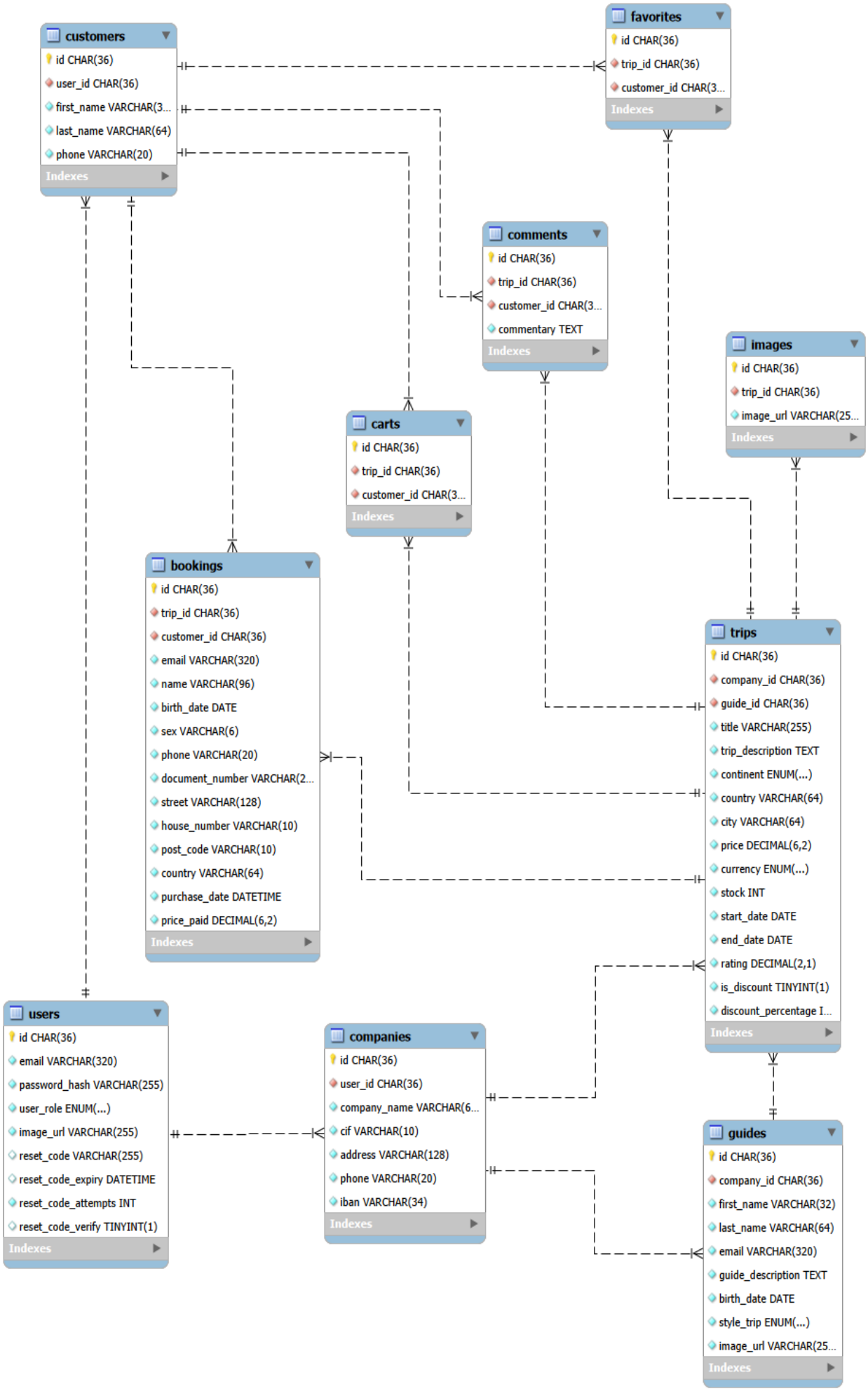
1. Capa de Decisión Inicial: El sistema verifica el estado de autenticación. Aunque el catálogo es público (lectura), las acciones de escritura (favoritos) y transaccionales (compras) están protegidas.
2. Interacción con la API: Cuando el usuario decide comprar o marcar un favorito, el Frontend (React) realiza una petición asíncrona. Si no hay un token JWT válido, Spring Security intercepta la petición y redirige al flujo de login.
3. Persistencia: Una vez validada la operación en el Backend, se ejecutan las transacciones en MySQL para asegurar que la compra quede registrada y el stock de plazas del paquete turístico se actualice correctamente.
4. Cierre del Ciclo: El flujo termina con una respuesta positiva del servidor que React renderiza como una confirmación visual para el cliente.

Diseño de la Base de Datos

El diseño de la base de datos constituye uno de los pilares fundamentales de la aplicación AventuraTrips. Se ha optado por un modelo relacional implementado sobre MySQL 8.4, que permite representar de manera fiel y eficiente todas las entidades del dominio y sus interrelaciones. A continuación se describe el esquema de la base de datos, las tablas que lo componen, sus atributos y las relaciones entre ellas.

Diagrama Entidad-Relación

A continuación se muestra el diagrama del esquema relacional de la base de datos, que representa todas las tablas, sus campos y las relaciones entre ellas mediante claves foráneas:


customers
id CHAR(36)
user_id CHAR(36)
first_name VARCHAR(3...
last_name VARCHAR(64)
phone VARCHAR(20)
Indexes
favorites
id CHAR(36)
trip_id CHAR(36)
customer_id CHAR(3...
Indexes
comments
id CHAR(36)
trip_id CHAR(36)
customer_id CHAR(3...
commentary TEXT
Indexes
images
id CHAR(36)
trip_id CHAR(36)
image_url VARCHAR(25...
Indexes
carts
id CHAR(36)
trip_id CHAR(36)
customer_id CHAR(3...
Indexes
bookings
id CHAR(36)
trip_id CHAR(36)
customer_id CHAR(36)
email VARCHAR(320)
name VARCHAR(96)
birth_date DATE
sex VARCHAR(6)
phone VARCHAR(20)
document_number VARCHAR(2...
street VARCHAR(128)
house_number VARCHAR(10)
post_code VARCHAR(10)
country VARCHAR(64)
purchase_date DATETIME
price_paid DECIMAL(6,2)
Indexes
trips
id CHAR(36)
company_id CHAR(36)
guide_id CHAR(36)
title VARCHAR(255)
trip_description TEXT
continent ENUM(...)
country VARCHAR(64)
city VARCHAR(64)
price DECIMAL(6,2)
currency ENUM(...)
stock INT
start_date DATE
end_date DATE
rating DECIMAL(2,1)
is_discount TINYINT(1)
discount_percentage I...
Indexes
users
id CHAR(36)
email VARCHAR(320)
password_hash VARCHAR(255)
user_role ENUM(...)
image_url VARCHAR(255)
reset_code VARCHAR(255)
reset_code_expiry DATETIME
reset_code_attempts INT
reset_code_verify TINYINT(1)
Indexes
companies
id CHAR(36)
user_id CHAR(36)
company_name VARCHAR(6...
cif VARCHAR(10)
address VARCHAR(128)
phone VARCHAR(20)
iban VARCHAR(34)
Indexes
guides
id CHAR(36)
company_id CHAR(36)
first_name VARCHAR(32)
last_name VARCHAR(64)
email VARCHAR(320)
guide_description TEXT
birth_date DATE
style_trip ENUM(...)
image_url VARCHAR(25...
Indexes

Descripción de las Tablas

Tabla users

La tabla users es la tabla central del sistema de autenticación. Almacena las credenciales de acceso y la información básica de seguridad de todos los usuarios registrados en la plataforma, independientemente de su rol. Cada registro contiene un identificador único (UUID), el correo electrónico (que actúa como nombre de usuario y es único en el sistema), el hash de la contraseña cifrado mediante BCrypt, el rol del usuario (CUSTOMER o COMPANY), la URL de la imagen de perfil almacenada en Cloudinary, y los campos necesarios para el flujo de recuperación de contraseña: el código de reseteo hasheado, su fecha de expiración, el contador de intentos fallidos y un indicador de verificación exitosa.

Tabla customers

La tabla customers almacena la información de perfil de los usuarios con rol de cliente. Mantiene una relación uno a uno con la tabla users mediante la clave foránea user_id (con restricción UNIQUE y CASCADE en eliminación). Sus campos incluyen el nombre, los apellidos y el número de teléfono del cliente, siendo este último único en el sistema para evitar duplicidades. Esta tabla es referenciada por las tablas bookings, carts, favorites y comments, estableciendo así las relaciones que permiten al cliente interactuar con los viajes.

Tabla companies

La tabla companies representa a las agencias o empresas turísticas registradas en la plataforma. Al igual que customers, mantiene una relación uno a uno con users. Sus campos reflejan la información corporativa necesaria: el nombre de la empresa, el CIF (Código de Identificación Fiscal), la dirección postal, el teléfono de contacto y el IBAN bancario. Todos estos campos cuentan con restricciones de unicidad para garantizar que no existan empresas duplicadas. Esta tabla es referenciada por trips y guides, ya que las empresas son las propietarias de los viajes y los guías turísticos.

Tabla guides

La tabla guides almacena la información de los guías turísticos asociados a cada empresa. Cada guía pertenece a una única empresa (relación muchos a uno con companies mediante company_id). Los campos incluyen el nombre completo, el correo electrónico (único), una descripción textual del guía, su fecha de nacimiento (utilizada para calcular la edad dinámicamente y validar que sea mayor de 18 años), el estilo de viaje en el que se especializa (RELAX, URBAN, ADVENTURE o LUXURY) y la URL de su fotografía almacenada en Cloudinary. Los guías son asignados a los viajes, estableciendo una relación uno a muchos con la tabla trips.

Tabla trips

La tabla trips es la entidad principal del catálogo de productos. Representa cada paquete turístico ofertado en la plataforma y contiene toda la información necesaria para su visualización y gestión comercial. Sus campos incluyen: el título del viaje, una descripción detallada (tipo TEXT), el continente y país-ciudad de destino, el precio con su divisa correspondiente (EUR, GBP, USD, CHF o CNY), el stock de plazas disponibles, las fechas de inicio y fin del viaje, la valoración (rating) generada automáticamente en un rango de 4.0 a 5.0, y los campos de descuento (indicador booleano y porcentaje). Cada viaje pertenece a una empresa (company_id) y tiene asignado un guía (guide_id). La tabla es referenciada por images, bookings, carts, favorites y comments.

Tabla images

La tabla images almacena las URLs de las imágenes asociadas a cada viaje. Cada viaje puede tener entre 1 y 5 imágenes, almacenadas como URLs seguras (HTTPS) de Cloudinary. La relación con trips es de muchos a uno mediante trip_id, con eliminación en cascada (CASCADE), lo que significa que al eliminar un viaje se eliminan automáticamente todos sus registros de imágenes asociados. Esta decisión de diseño simplifica la gestión del ciclo de vida de las imágenes y mantiene la integridad referencial.

Tabla bookings

La tabla bookings registra todas las reservas realizadas por los clientes. Es la tabla transaccional principal del sistema y almacena no solo la referencia al viaje y al cliente, sino también una copia completa de los datos de facturación proporcionados durante el checkout: correo electrónico, nombre completo, fecha de nacimiento, sexo, teléfono, número de documento de identidad, y dirección postal completa (calle, número, código postal y país). Además, registra la fecha y hora exacta de la compra y el precio pagado en el momento de la transacción. Se aplica una restricción compuesta única sobre (customer_id, trip_id) para evitar que un mismo cliente reserve el mismo viaje más de una vez. El sistema también valida que no existan reservas duplicadas por número de documento para un mismo viaje, independientemente del cliente.

Tabla carts

La tabla carts implementa el carrito de compras de cada cliente. Su estructura es intencionalmente simple, conteniendo únicamente el identificador del cliente y del viaje. Se aplica una restricción compuesta única sobre (customer_id, trip_id) que impide que un cliente añada el mismo viaje al carrito más de una vez. Los registros del carrito se eliminan automáticamente cuando el cliente completa el proceso de checkout, tanto para los viajes comprados como para aquellos que también estaban en su lista de favoritos.

Tabla favorites

La tabla favorites funciona como la lista de deseos o wishlist del cliente. Su estructura es idéntica a la del carrito: contiene la referencia al cliente y al viaje, con una restricción compuesta única sobre (customer_id, trip_id). Permite a los clientes marcar viajes de interés para consultarlos posteriormente. Al igual que los elementos del carrito, los favoritos correspondientes a viajes comprados se limpian automáticamente tras completar el checkout.

Tabla comments

La tabla comments está diseñada para almacenar comentarios o reseñas de los clientes sobre los viajes. Contiene la referencia al viaje, al cliente y el texto del comentario. Aunque la tabla y su entidad están definidas en el modelo de datos, esta funcionalidad no se ha implementado completamente en la versión actual de la aplicación, quedando como una extensión prevista para futuras iteraciones del proyecto.

Relaciones y Cardinalidades

El esquema relacional de AventuraTrips implementa las siguientes relaciones principales entre sus entidades:

• User - Customer (1:1): Cada usuario con rol CUSTOMER tiene exactamente un registro en la tabla customers. La eliminación del usuario propaga en cascada la eliminación del cliente.

• User - Company (1:1): Cada usuario con rol COMPANY tiene exactamente un registro en la tabla companies. Misma política de cascada que la relación anterior.

• Company - Guide (1:N): Una empresa puede tener múltiples guías turísticos. Cada guía pertenece a una única empresa.

• Company - Trip (1:N): Una empresa puede ofertar múltiples viajes. Cada viaje pertenece a una única empresa.

• Guide - Trip (1:N): Un guía puede estar asignado a múltiples viajes. Cada viaje tiene asignado un único guía.

• Trip - Image (1:N): Un viaje puede tener entre 1 y 5 imágenes. Las imágenes se eliminan en cascada con el viaje.

• Customer - Booking (1:N): Un cliente puede realizar múltiples reservas de viajes diferentes.

• Customer - Cart (1:N): Un cliente puede tener múltiples viajes en su carrito de compras.

• Customer - Favorite (1:N): Un cliente puede marcar múltiples viajes como favoritos.

• Trip - Booking (1:N): Un viaje puede ser reservado por múltiples clientes diferentes.

Estrategia de Claves Primarias e Indexación

Todas las tablas utilizan UUID (Universally Unique Identifier) como clave primaria, almacenado como VARCHAR(36). Esta decisión de diseño se fundamenta en varias razones: los UUID eliminan la posibilidad de colisiones al generar identificadores de forma distribuida, no revelan información sobre el volumen de datos o el orden de inserción (a diferencia de los identificadores auto-incrementales), y facilitan la integración con sistemas externos al ser identificadores universalmente únicos.

En cuanto a la indexación, el esquema aprovecha los índices implícitos generados por las restricciones de clave primaria (PRIMARY KEY), unicidad (UNIQUE) y clave foránea (FOREIGN KEY). Los campos con restricción UNIQUE como el correo electrónico, teléfono, nombre de empresa, CIF e IBAN cuentan con índices que optimizan las búsquedas y validaciones de duplicados. Las claves foráneas generan índices que mejoran el rendimiento de las consultas JOIN. Además, las restricciones compuestas únicas en las tablas carts y favorites actúan como índices compuestos que optimizan las consultas de verificación de duplicados.

Tipos de Datos y Restricciones

El esquema emplea tipos de datos específicos para cada campo según su naturaleza y requisitos de precisión:

• VARCHAR(36) para identificadores UUID.

• VARCHAR(320) para correos electrónicos, siguiendo la especificación RFC 5321 que establece una longitud máxima de 320 caracteres para direcciones de correo.

• VARCHAR(255) para hashes de contraseñas BCrypt y URLs de imágenes.

• DECIMAL(6,2) para precios, permitiendo valores hasta 9.999,99 con precisión de dos decimales.

• DECIMAL(2,1) para valoraciones (ratings), permitiendo valores de 0,0 a 9,9 con un decimal.

• TEXT para campos de descripción que pueden contener textos extensos sin límite fijo.

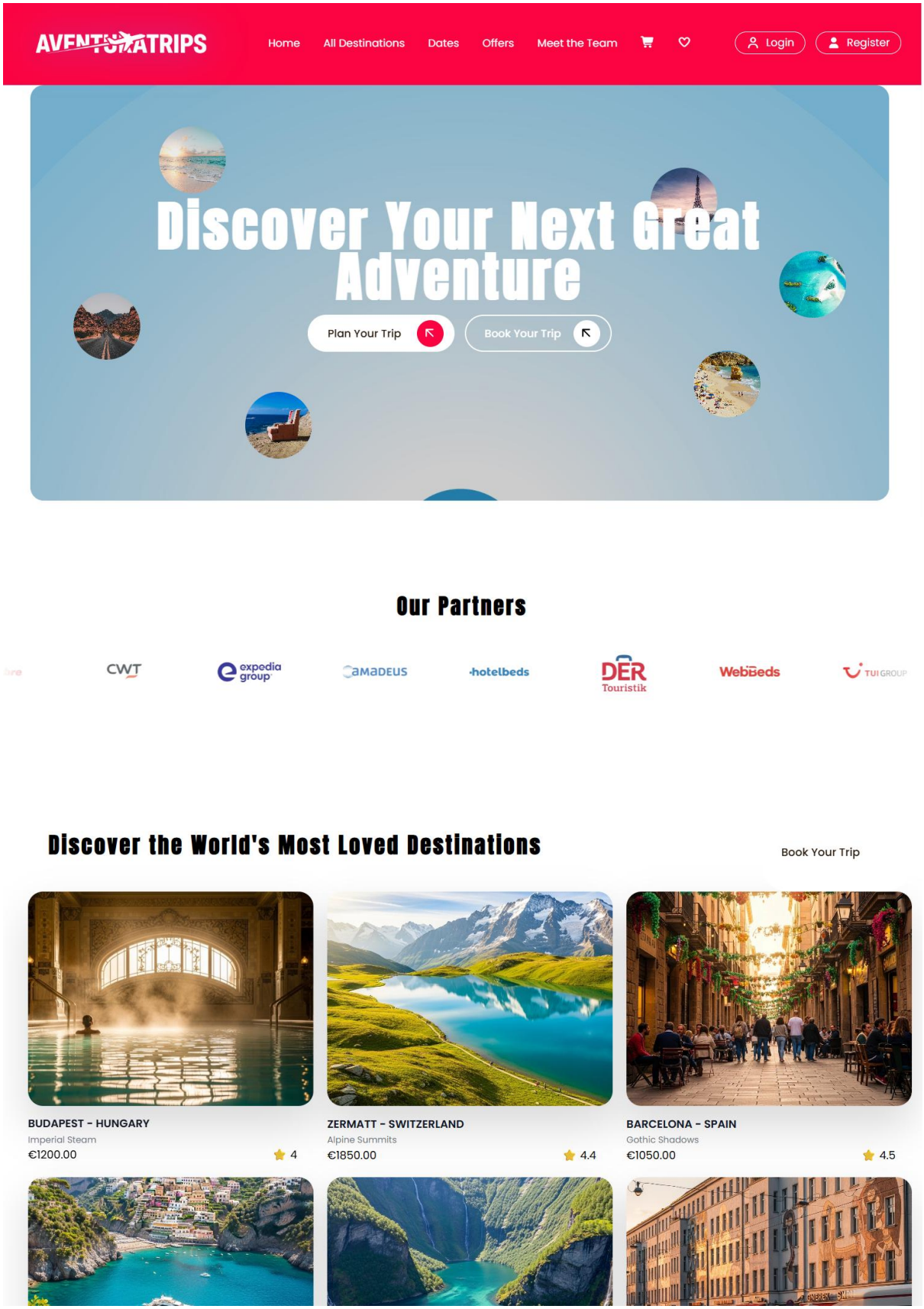
• DATE para fechas (nacimiento, inicio y fin de viaje) y DATETIME para marcas de tiempo de transacciones.

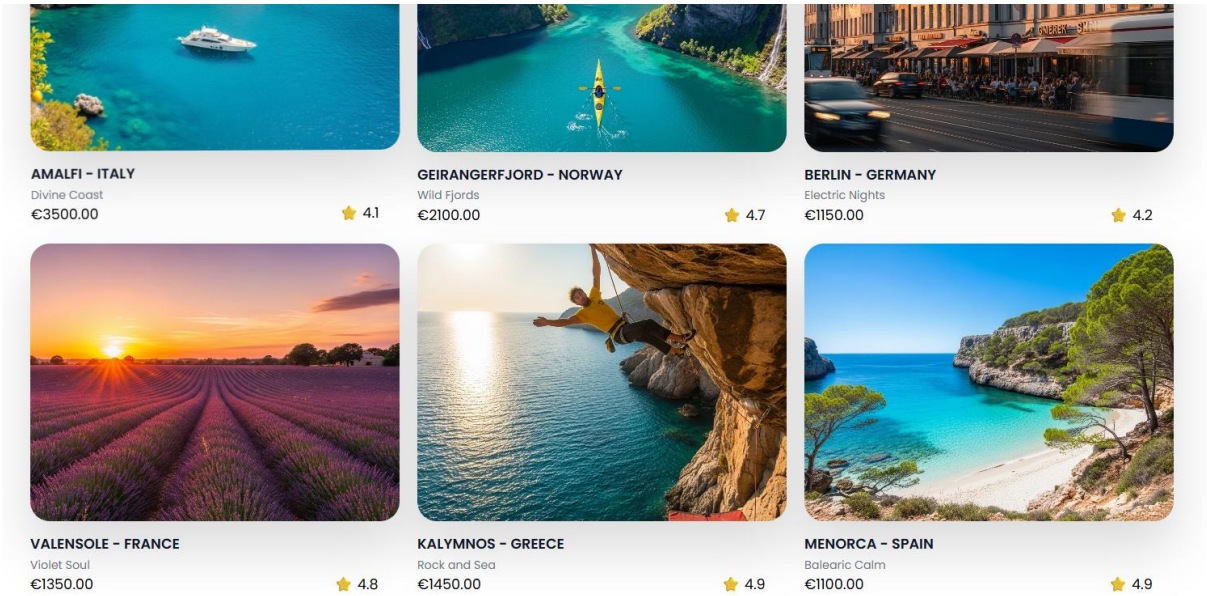
• BOOLEAN para indicadores binarios como is_discount y reset_code_verify.

• INT para valores numéricos enteros como stock, porcentaje de descuento e intentos de reseteo.

Diseño de Interfaces

Versión Escritorio


AVENTURATRIPS
Home
All Destinations
Dates
Offers
Meet the Team
Login
Register
Discover Your Next Great Adventure
Plan Your Trip
Book Your Trip
Our Partners
CWT
expedia group
amadeus
hotelbeds
DER Touristik
WebBeds
TUI GROUP
Discover the World's Most Loved Destinations
Book Your Trip
BUDAPEST - HUNGARY
Imperial Steam
€1200.00
4
ZERMATT - SWITZERLAND
Alpine Summits
€1850.00
4.4
BARCELONA - SPAIN
Gothic Shadows
€1050.00
4.5


AMALFI - ITALY
Divine Coast
€3500.00
4.1
GEIRANGERFJORD - NORWAY
Wild Fjords
€2100.00
4.7
BERLIN - GERMANY
Electric Nights
€1150.00
4.2
VALENSOLE - FRANCE
Violet Soul
€1350.00
4.8
KALYMNOS - GREECE
Rock and Sea
€1450.00
4.9
MENORCA - SPAIN
Balearic Calm
€1100.00
4.9

Plan Your Trip in 3 Steps

1

CHOOSE DESTINATION

Pick a place that excites you — from cities to urban area's hidden gems.

2

CUSTOMIZE YOUR EXPERIENCE

Select activities, stays, and add-ons to match your own style.

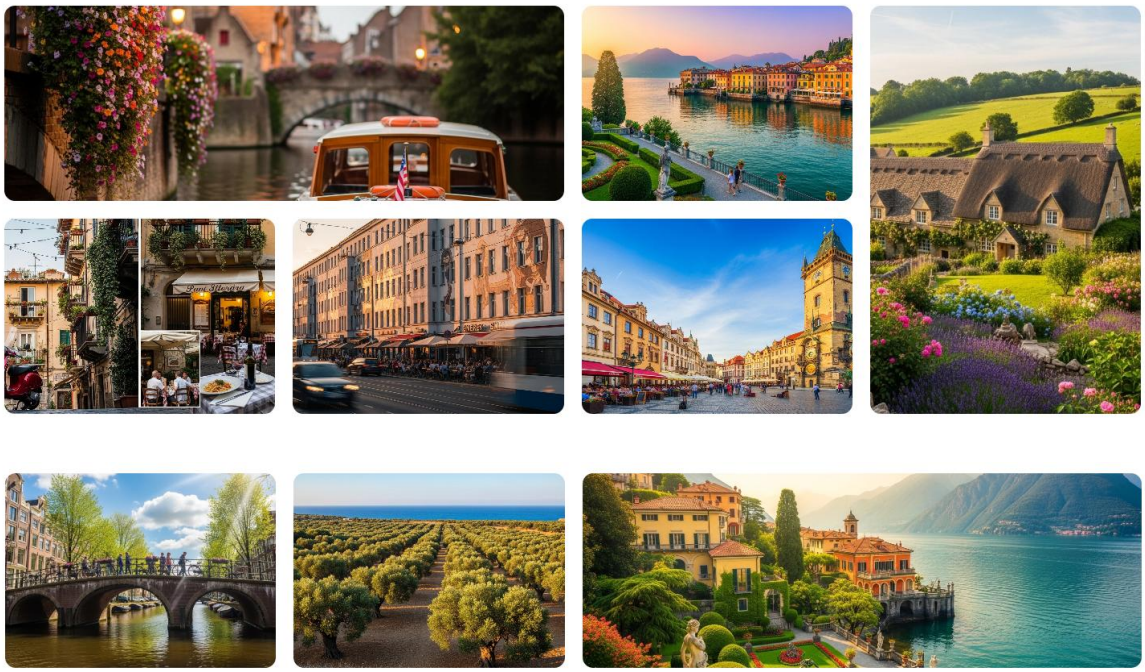
3

BOOK & GO

Confirm your trip in minutes and start the countdown.

Our Visual Journey Through UNFORGETTABLE DESTINATIONS

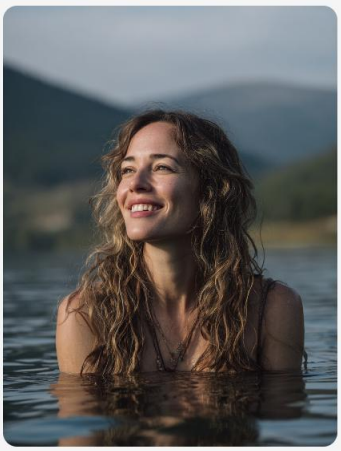
Book Your Trip



Rated Excellent by Over
500K HAPPY GLOBAL TRAVELERS

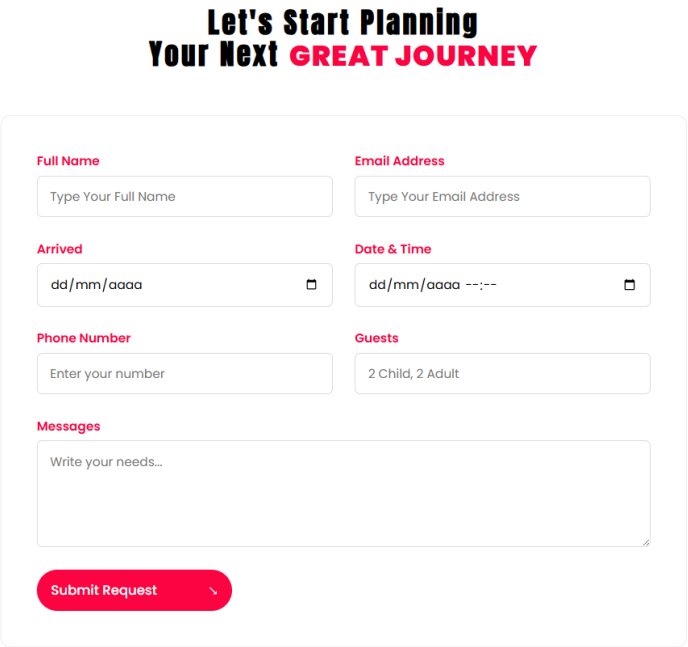
4.5K	30+	150+	4.7
Trips Booked	Destinations Covered	Verified Stays	Customer Rating

Our Guides

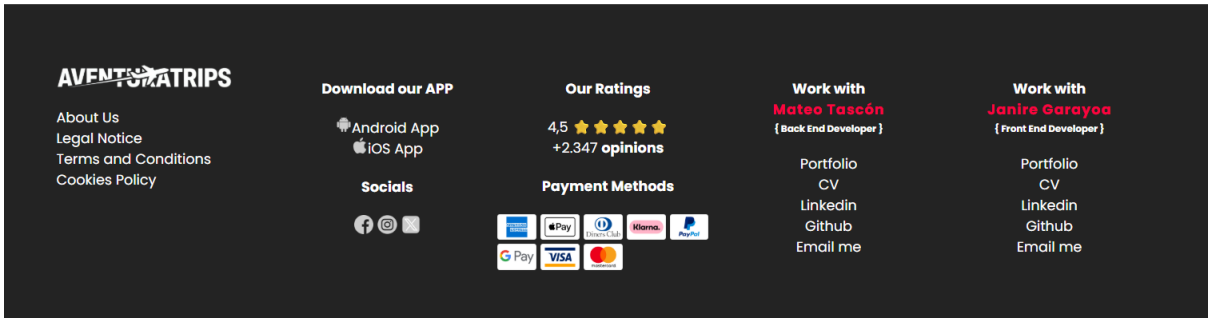


Valentina Reyes

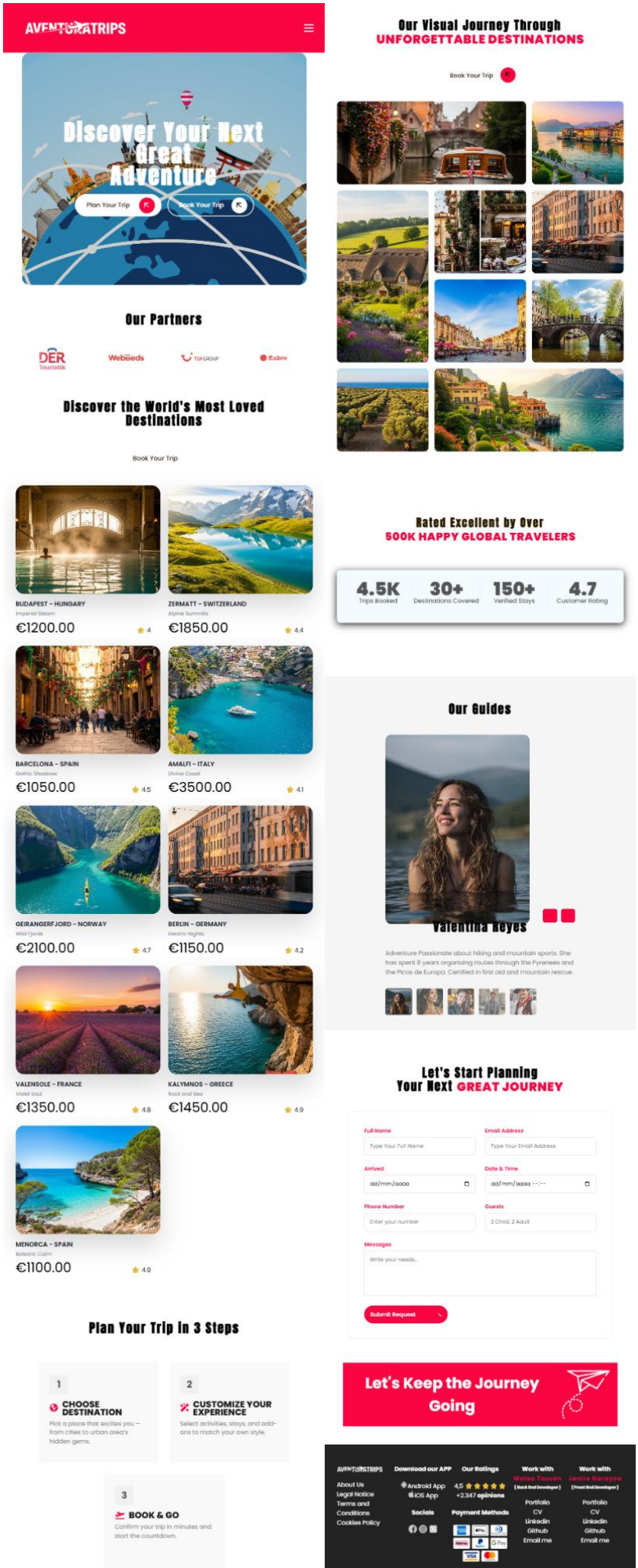
Adventure Passionate about hiking and mountain sports. She has spent 8 years organizing routes through the Pyrenees and the Picos de Europa. Certified in first aid and mountain rescue.


Let's Start Planning
Your Next GREAT JOURNEY
Full Name
Type Your Full Name
Email Address
Type Your Email Address
Arrived
dd/mm/aaaa
Date & Time
dd/mm/aaaa --:--
Phone Number
Enter your number
Guests
2 Child, 2 Adult
Messages
Write your needs...
Submit Request

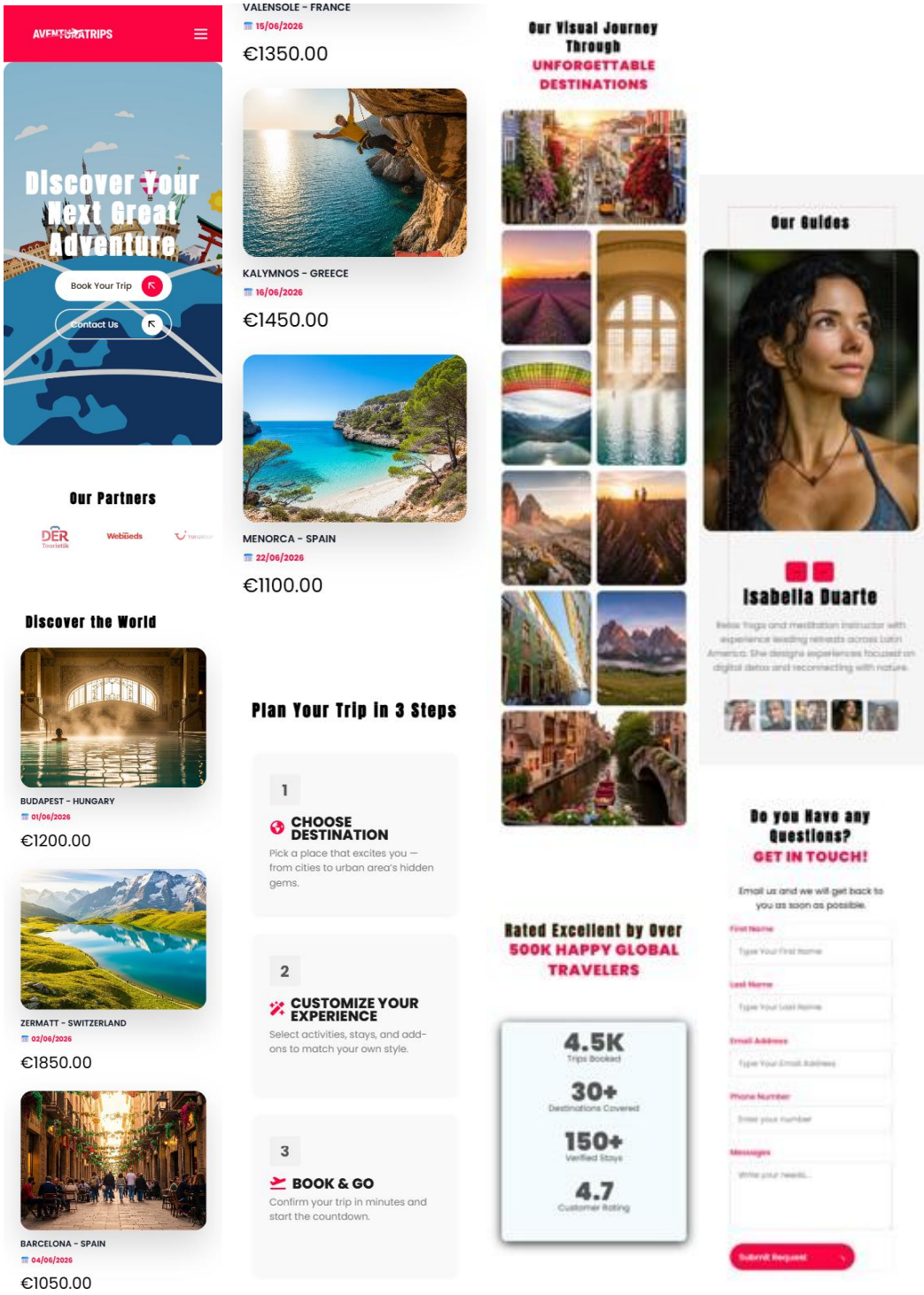
Let's Keep the Journey Going

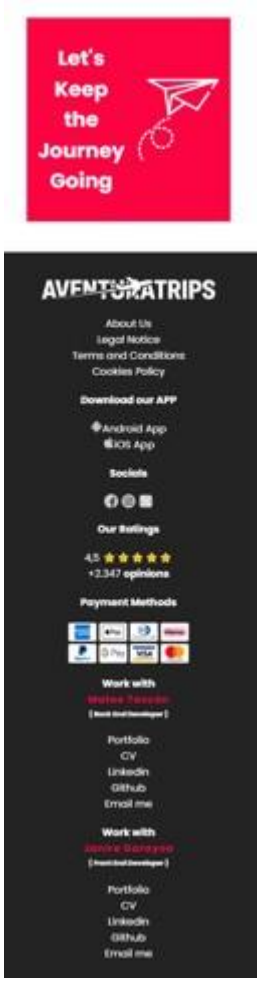

AVENTURATRIPS
About Us
Legal Notice
Terms and Conditions
Cookies Policy
Download our APP
Android App
iOS App
Socials
Our Ratings
4,5
+2.347 opinions
Payment Methods
Work with
Mateo Tascón
{ Back End Developer }
Portfolio
CV
Linkedin
Github
Email me
Work with
Janire Garayoa
{ Front End Developer }
Portfolio
CV
Linkedin
Github
Email me

Versión Tablet


AVENTURATRIPS
Discover Your Next Great Adventure
Plan Your Trip
Book Your Trip
Our Partners
DER Touristik
Webbeds
TUI GROUP
Sabre
Discover the World's Most Loved Destinations
Book Your Trip
BUDAPEST - HUNGARY
Imperial Steam
€1200.00
4
ZERMATT - SWITZERLAND
Alpine Summits
€1850.00
4.4
BARCELONA - SPAIN
Gothic Shadows
€1050.00
4.5
AMALFI - ITALY
Divine Coast
€3500.00
4.1
GEIRANGERFJORD - NORWAY
Wild Fjords
€2100.00
4.7
BERLIN - GERMANY
Electric Nights
€1150.00
4.2
VALENSOLE - FRANCE
Violet Soul
€1350.00
4.8
KALYMNOS - GREECE
Rock and Sea
€1450.00
4.9
MENORCA - SPAIN
Balearic Calm
€1100.00
4.9
Plan Your Trip in 3 Steps
1
CHOOSE DESTINATION
Pick a place that excites you — from cities to urban area's hidden gems.
2
CUSTOMIZE YOUR EXPERIENCE
Select activities, stays, and add-ons to match your own style.
3
BOOK & GO
Confirm your trip in minutes and start the countdown.
Our Visual Journey Through UNFORGETTABLE DESTINATIONS
Book Your Trip
Rated Excellent by Over 500K HAPPY GLOBAL TRAVELERS
4.5K Trips Booked
30+ Destinations Covered
150+ Verified Stays
4.7 Customer Rating
Our Guides
Valentina Reyes
Adventure Passionate about hiking and mountain sports. She has spent 8 years organizing routes through the Pyrenees and the Picos de Europa. Certified in first aid and mountain rescue.
Let's Start Planning Your Next GREAT JOURNEY
Full Name
Type Your Full Name
Email Address
Type Your Email Address
Arrived
dd/mm/aaaa
Date & Time
dd/mm/aaaa --:--
Phone Number
Enter your number
Guests
2 Child, 2 Adult
Messages
Write your needs...
Submit Request
Let's Keep the Journey Going
About Us
Legal Notice
Terms and Conditions
Cookies Policy
Download our APP
Android App
iOS App
Our Ratings
4,5
+2.347 opinions
Socials
Payment Methods
Work with
Mateo Tasada
(Back End Developer)
Portfolio
CV
Linkedin
Github
Email me
Work with
Jonora Garayoa
(Front End Developer)
Portfolio
CV
Linkedin
Github
Email me

Versión Móvil


AVENTURATRIPS
Discover Your Next Great Adventure
Book Your Trip
Contact Us
Our Partners
Discover the World
BUDAPEST - HUNGARY
01/06/2026
€1200.00
ZERMATT - SWITZERLAND
02/06/2026
€1850.00
BARCELONA - SPAIN
04/06/2026
€1050.00
VALENSOLE - FRANCE
15/06/2026
€1350.00
KALYMNOS - GREECE
16/06/2026
€1450.00
MENORCA - SPAIN
22/06/2026
€1100.00
Plan Your Trip in 3 Steps
1
CHOOSE DESTINATION
Pick a place that excites you — from cities to urban area's hidden gems.
2
CUSTOMIZE YOUR EXPERIENCE
Select activities, stays, and add-ons to match your own style.
3
BOOK & GO
Confirm your trip in minutes and start the countdown.
Our Visual Journey Through UNFORGETTABLE DESTINATIONS
Rated Excellent by Over 500K HAPPY GLOBAL TRAVELERS
4.5K
Trips Booked
30+
Destinations Covered
150+
Verified Stays
4.7
Customer Rating
Our Guides
Isabella Duarte
Do you Have any Questions?
GET IN TOUCH!
Email us and we will get back to you as soon as possible.
First Name
Last Name
Email Address
Phone Number
Messages
Submit Request



Tecnologías y Herramientas

Para el desarrollo de AventuraTrips, se ha seleccionado un stack tecnológico moderno, robusto y ampliamente demandado en el sector laboral. La elección de estas herramientas busca maximizar el rendimiento, la escalabilidad y la eficiencia en el ciclo de vida del desarrollo.

Tecnologías Frontend

- React: Librería de JavaScript de código abierto orientada a la creación de interfaces de usuario basadas en componentes. Se ha utilizado para gestionar un estado complejo y garantizar una experiencia de usuario (UX) fluida y reactiva.

- Se diseñó un Loader animado mediante CSS3 para homogeneizar la espera del usuario mientras se recuperan datos de la API, evitando saltos visuales (Layout Shift).
- Vite: Herramienta de construcción de próxima generación. A diferencia de las herramientas tradicionales, Vite ofrece un servidor de desarrollo extremadamente rápido gracias al uso de *Native ES Modules*, optimizando los tiempos de recarga y el empaquetado final del proyecto.

Tecnologías Backend

Java 21: Lenguaje de programación orientado a objetos utilizado como base del backend. La versión 21, siendo una versión LTS (Long Term Support), incorpora mejoras de rendimiento, records, pattern matching y otras características modernas del lenguaje que facilitan la escritura de código limpio y eficiente.

Spring Boot 4.0.1: Framework de desarrollo que simplifica la configuración y el despliegue de aplicaciones basadas en Spring. Proporciona auto-configuración, un servidor embebido Tomcat, gestión de dependencias mediante starters y un ecosistema completo para el desarrollo de APIs REST empresariales. Su enfoque de

"convention over configuration" reduce significativamente el boilerplate y acelera el desarrollo.

Spring Security: Módulo del ecosistema Spring que proporciona autenticación y autorización robustas. En AventuraTrips se ha configurado para trabajar con tokens JWT de forma stateless, definiendo reglas de acceso basadas en roles (CUSTOMER y COMPANY) que protegen los endpoints sensibles mientras mantienen accesibles los recursos públicos.

Spring Data JPA: Abstracción de acceso a datos que simplifica la interacción con la base de datos mediante el patrón Repository. Permite definir interfaces de repositorio con métodos de consulta derivados del nombre del método, además de soportar consultas nativas SQL y JPQL para operaciones más complejas como la paginación manual y el bloqueo pesimista.

JWT (JSON Web Token): Estándar abierto (RFC 7519) utilizado para la transmisión segura de información entre el cliente y el servidor. Los tokens se generan con el algoritmo HS512 y contienen claims con el identificador del usuario, su rol y la fecha de expiración, permitiendo una autenticación stateless que no requiere almacenamiento de sesión en el servidor.

Lombok: Biblioteca que reduce el código repetitivo (boilerplate) en Java mediante anotaciones que generan automáticamente getters, setters, constructores, builders y otros métodos estándar durante la compilación. Su uso mejora la legibilidad del código y acelera el desarrollo.

Maven: Herramienta de gestión de proyectos y automatización de la compilación. Gestiona las dependencias del proyecto, el ciclo de vida de construcción (compilación, empaquetado, despliegue) y la generación de artefactos ejecutables JAR.

Swagger/OpenAPI (SpringDoc): Herramienta de documentación interactiva de la API REST. Genera automáticamente una interfaz web que permite visualizar y probar todos los endpoints de la API, incluyendo los esquemas de petición y respuesta, los códigos de estado HTTP y los requisitos de autenticación.

Gestión de Datos y Almacenamiento

MySQL 8.4: Sistema de gestión de bases de datos relacional utilizado para la persistencia de toda la información de la aplicación. MySQL ha sido seleccionado por su robustez, su amplio soporte en la comunidad de desarrollo, su compatibilidad con Spring Data JPA y Hibernate, y su rendimiento probado en aplicaciones web de

producción. La base de datos almacena 10 tablas que modelan las entidades del dominio y sus relaciones.

Cloudinary: Plataforma de gestión de activos multimedia en la nube utilizada para el almacenamiento, la transformación y la distribución de todas las imágenes de la aplicación (fotografías de viajes, imágenes de guías y fotos de perfil de usuarios). La integración se realiza mediante el SDK oficial de Cloudinary para Java, que proporciona métodos para la carga y eliminación de archivos. Todas las imágenes se almacenan en una carpeta dedicada (tfg/) dentro del espacio de Cloudinary, y se sirven a través de URLs seguras HTTPS con CDN global para garantizar tiempos de carga óptimos.

Resend: Servicio de correo electrónico transaccional utilizado para el envío de notificaciones automáticas a los usuarios. A través de su API REST y SDK para Java, la aplicación envía cuatro tipos de correos: códigos de recuperación de contraseña, recibos de compra con el detalle de los viajes adquiridos, confirmaciones de cancelación de reservas con información del reembolso, y notificaciones de formularios de contacto enviadas por visitantes de la web.

Docker y Docker Compose: Tecnologías de contenerización utilizadas para empaquetar y desplegar tanto la aplicación Spring Boot como la base de datos MySQL en contenedores aislados y reproducibles. El archivo docker-compose.yml

define dos servicios: el contenedor MySQL (con volumen persistente para los datos) y el contenedor de la aplicación Java, que se conecta a la base de datos mediante variables de entorno configurables. Esta aproximación garantiza que el entorno de ejecución sea idéntico en desarrollo y producción.

Herramientas de Desarrollo (Tooling)

- Visual Studio Code (VS Code): Entorno de desarrollo integrado (IDE) principal para el frontend.
- IntelliJ IDEA: IDE especializado en Java utilizado para el desarrollo y depuración del backend.
- Postman: Herramienta fundamental para el testeo y validación de los endpoints de la API REST antes de su integración con el frontend.
- Git & GitHub: Control de versiones para el seguimiento del código y la colaboración durante el desarrollo del proyecto.

Metodología

Metodología empleada y justificación

Para el desarrollo de AventuraTrips hemos optado por una metodología iterativo-incremental con elementos de Scrum ligero, adaptada al tamaño reducido del equipo (dos personas) y al carácter académico del proyecto.

La decisión se fundamenta en varios criterios. En primer lugar, el alcance del proyecto no estaba completamente cerrado al inicio: durante la reunión de seguimiento del 17

de febrero de 2026 se redefinieron varios requisitos (foto de perfil para todos los usuarios, asignación obligatoria de un guía por viaje y modelo basado en una sola empresa) que obligaron a refactorizar parte del backend ya implementado. En segundo lugar, la división natural del trabajo (un miembro en backend y otro en frontend) requería sincronización frecuente pero no justificaba la adopción de un marco formal pesado como Scrum clásico. Por último, las entregas parciales nos permitían validar funcionalidades de forma progresiva antes de integrarlas, reduciendo el riesgo de acumular errores al final del proyecto.

El marco concreto que hemos seguido se apoya en tres pilares:

Iteraciones semanales. Al comienzo de cada semana se definían las funcionalidades objetivo (equivalentes a sprints cortos) y al finalizar se realizaba una revisión conjunta. Las iteraciones no tenían duración rígida: algunas fueron de 5 días y otras de 10 en función de la complejidad del módulo a implementar.

Reuniones de sincronización. Además del contacto diario informal, mantuvimos reuniones formales cuando se cerraba un bloque funcional importante o cuando surgía una decisión de diseño que afectaba a ambas capas. La reunión del 17/02/2026 es el ejemplo más visible, con impacto directo en el modelo de datos y en el alcance del proyecto.

Commits como registro de progreso. Cada commit representa una unidad de trabajo funcional completada. El mensaje de commit describe la funcionalidad añadida o el problema resuelto, lo que nos ha permitido reconstruir el cronograma real del proyecto con precisión.

Este enfoque nos ha permitido adaptarnos a los cambios (el más significativo, la redefinición de alcance de febrero) sin tener que replanificar desde cero, aprovechando el trabajo ya hecho y reorientando únicamente lo necesario.

Planificación inicial vs real

Al comienzo del proyecto, durante la redacción del anteproyecto en diciembre de 2025, se elaboró una estimación inicial de las horas necesarias por bloque funcional (RFTP).

La planificación real se ha reconstruido a partir del registro de actividad diaria (Toggl) contrastado con el historial de commits de Git, que documenta de forma verificable las fechas y los entregables de cada fase. La siguiente figura muestra la comparación entre ambas planificaciones, con indicación explícita de las desviaciones por bloque.

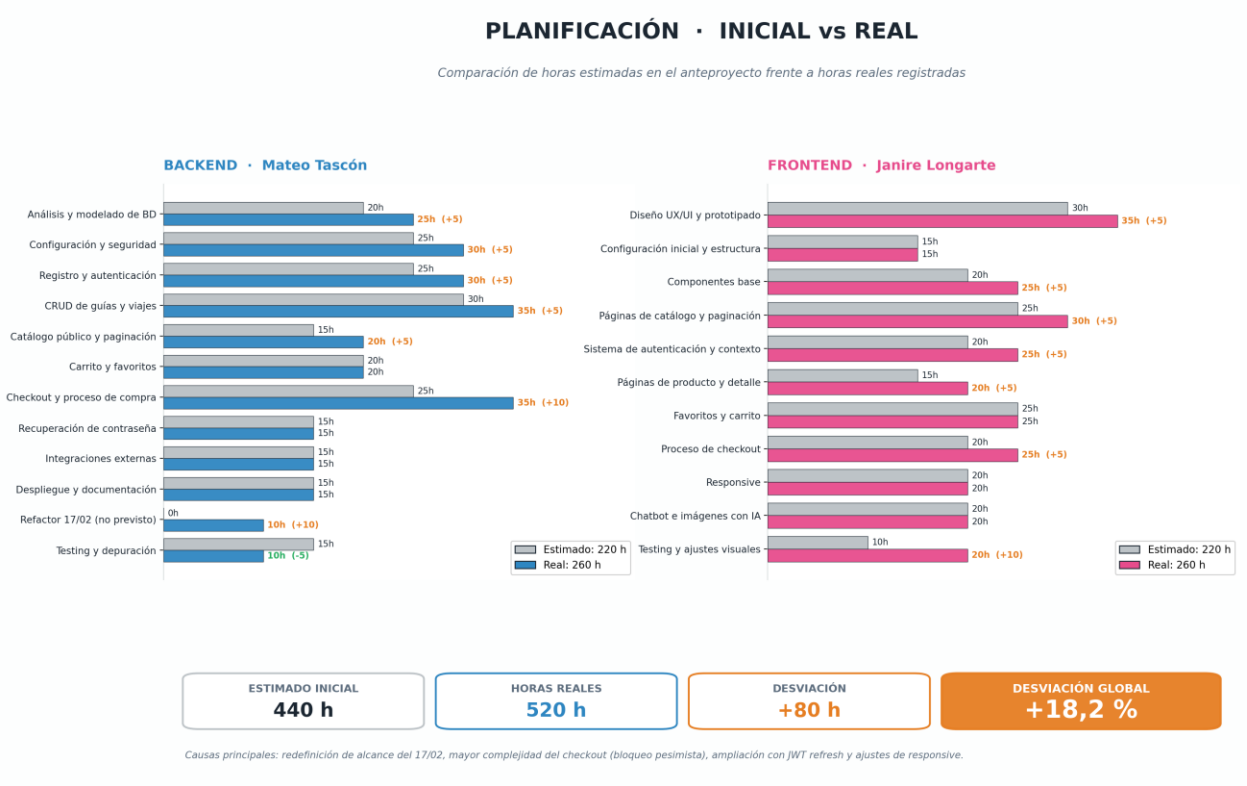


Figura 1. Comparación de la planificación inicial frente a la real. 440 h estimadas frente a 520 h reales, con una desviación global del +18,2 %.

Análisis de desviaciones

Comparando la planificación inicial (440 horas) con la real (520 horas) se observa una desviación global del +18,2%, equivalente a 80 horas adicionales repartidas entre ambos miembros del equipo. Las causas principales son las siguientes:

Redefinición de alcance a mitad del proyecto (17/02/2026). La reunión de seguimiento mantenida el 17 de febrero de 2026 introdujo tres cambios no previstos: la incorporación de una imagen de perfil para todos los usuarios, la asignación obligatoria de un guía a cada viaje (relación 1:N Guide-Trip) y la redefinición del modelo de dominio para

soportar una única empresa en lugar de varias. Estos cambios obligaron a refactorizar entidades ya implementadas y han supuesto aproximadamente 10 horas adicionales en backend.

Infraestimación del checkout. La implementación del proceso de compra resultó más compleja de lo previsto por la necesidad de garantizar la concurrencia mediante bloqueo pesimista (SELECT ... FOR UPDATE), la validación de duplicados por número de documento y la integración con el envío de recibos por correo electrónico. Se han dedicado 10 horas adicionales respecto a la estimación inicial.

Ampliación del sistema de autenticación. Se añadió un mecanismo de renovación automática del JWT (auto-refresh en los últimos 15 minutos de vida) no contemplado inicialmente, documentado en el commit del 07/04/2026 en el frontend, que supuso unas 5 horas adicionales.

Ajustes de responsive y testing visual. El testing manual del frontend en distintos dispositivos requirió más iteraciones de las previstas, especialmente en la versión móvil, con un sobrecoste de 10 horas.

Valoración general. Una desviación del 18% sobre la planificación inicial se considera razonable en proyectos de desarrollo de software, donde la literatura especializada sitúa el margen típico de desviación entre el 15% y el 30%. El uso de una metodología iterativa nos ha permitido absorber los cambios sin necesidad de replanificar desde cero.

Diagrama de Gantt

El cronograma real del proyecto, contrastable con el historial de Git, se distribuye en las siguientes fases:

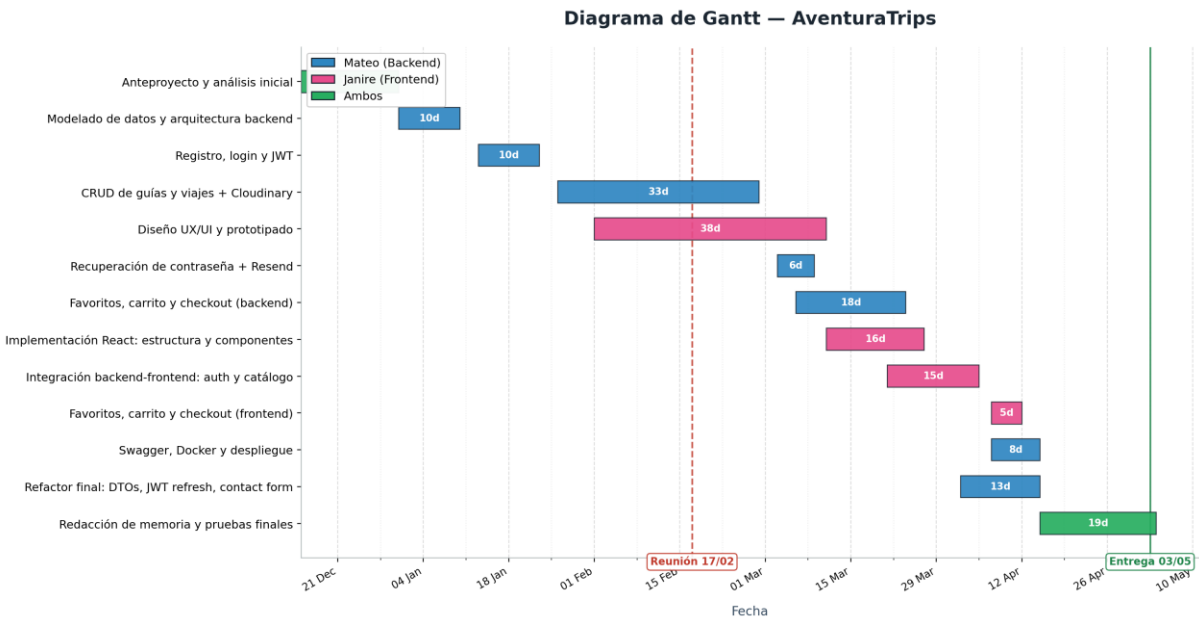


Figura 2. Diagrama de Gantt del proyecto AventuraTrips. Cronograma real contrastado con el historial de Git.

Presupuesto

El presupuesto del proyecto se calcula aplicando una tarifa de 30,00 € por hora (tarifa media de un desarrollador junior en el mercado español, 2026) a las horas reales dedicadas por cada miembro del equipo.

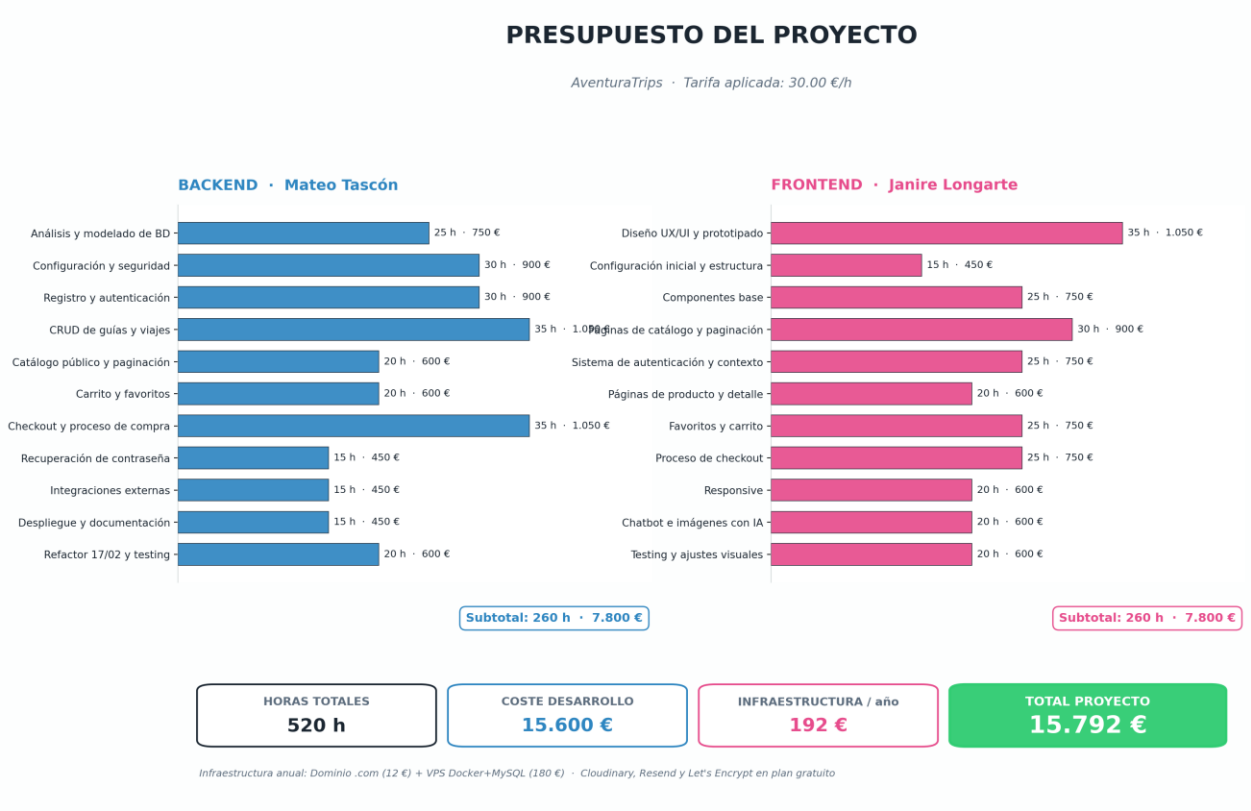


Figura 3. Presupuesto detallado de AventuraTrips. 520 horas totales de desarrollo a 30 €/h, con coste total del proyecto de 15.792 € (desarrollo + primer año de infraestructura).

README y control de versiones (GIT)

El proyecto se ha versionado íntegramente con Git y alojado en repositorios públicos de GitHub, separados por capa:

Repositorio del backend: contiene el código fuente Java/Spring Boot, los scripts de la base de datos, el fichero docker-compose.yml y la documentación de la API (API_DOCUMENTATION.md + Swagger).

Repositorio del frontend: contiene el código fuente React, los activos multimedia generados con IA y los archivos de configuración de Vite.

El flujo de trabajo seguido ha sido un modelo trunk-based simple: rama principal main y commits frecuentes tras cada unidad funcional completada. Al tratarse de un equipo de dos personas trabajando en módulos desacoplados (backend y frontend) no se han

necesitado ramas de larga duración ni pull requests formales. Los conflictos de integración se resolvieron en las sesiones de sincronización semanales.

Los mensajes de commit siguen un estilo descriptivo en inglés (backend) y en español (frontend), indicando la funcionalidad añadida o corregida. Esta disciplina ha sido clave para reconstruir la planificación real del proyecto, ya que cada commit actúa como punto de control verificable del progreso. El backend registra 56 commits repartidos en 34 días activos entre diciembre de 2025 y abril de 2026; el frontend registra 37 commits en GitHub, complementados con el trabajo previo realizado en local antes de la primera subida del 11/03/2026.

Cada repositorio incluye un archivo README que documenta: descripción general del módulo, requisitos previos (Java 21 y Maven para backend; Node.js y npm para frontend), pasos de instalación y ejecución local, variables de entorno necesarias (.env.example en backend) y enlaces a la documentación complementaria (Swagger y Notion).

Desarrollo del Backend

En esta sección se detalla el desarrollo completo del backend de AventuraTrips, abarcando la estructura del proyecto, la implementación de la seguridad, los endpoints de la API REST, la lógica de negocio principal y las integraciones con servicios externos.

Estructura del Proyecto

El backend de AventuraTrips sigue una arquitectura de capas (Layered Architecture) implementada mediante los paquetes de Java. Esta organización separa claramente las responsabilidades de cada componente del sistema:

• Paquete controllers: Contiene los cuatro controladores REST que definen los endpoints de la API. Cada controlador está anotado con @RestController y @RequestMapping para establecer la ruta base de sus endpoints. Los controladores reciben las peticiones HTTP, delegan la lógica de negocio a los servicios correspondientes y devuelven las respuestas encapsuladas en el objeto ApiResponse.

• Paquete services: Implementa toda la lógica de negocio de la aplicación. Se estructura mediante interfaces (en el subpaquete interfaces) y sus implementaciones (en el subpaquete classes), siguiendo el principio de inversión de dependencias. Los servicios son componentes transaccionales anotados con @Service y @Transactional que coordinan las operaciones entre repositorios, validaciones y servicios externos.

• Paquete repositories: Define las interfaces de acceso a datos que extienden JpaRepository de Spring Data JPA. Cada repositorio proporciona operaciones CRUD automáticas y métodos de consulta personalizados mediante la anotación @Query, incluyendo consultas nativas SQL para operaciones específicas como la paginación manual, la selección aleatoria y el bloqueo pesimista.

• Paquete models: Contiene las entidades JPA que representan las tablas de la base de datos. Cada entidad está anotada con @Entity y @Table, y define sus campos, relaciones y

restricciones mediante anotaciones de JPA como @OneToOne, @OneToMany, @ManyToOne, @Column y @UniqueConstraint.

• Paquete dto: Agrupa los objetos de transferencia de datos separados en dos subpaquetes: dto/in para los DTOs de entrada (peticiones) y dto/out para los DTOs de salida (respuestas). Los DTOs de entrada incorporan validaciones mediante anotaciones de Jakarta Validation como @NotBlank, @Size, @Email, @Pattern, @Positive, @Past y @Future.

• Paquete security: Implementa toda la infraestructura de seguridad JWT, incluyendo el servicio de generación y validación de tokens, el filtro de autenticación que intercepta las peticiones HTTP, y los manejadores de errores de autenticación y autorización.

• Paquete config: Contiene las clases de configuración de Spring que definen beans para los servicios externos (Cloudinary, Resend), la configuración de seguridad (SecurityConfig), la documentación OpenAPI y el manejador global de excepciones.

• Paquete exceptions: Define 21 excepciones personalizadas que cubren todos los escenarios de error de la aplicación, desde validaciones de datos hasta errores de integración con servicios externos.

Seguridad y Autenticación

La seguridad de AventuraTrips se implementa mediante Spring Security en combinación con JSON Web Tokens (JWT), siguiendo un modelo de autenticación stateless. Este enfoque elimina la necesidad de mantener sesiones en el servidor, lo que resulta especialmente adecuado para APIs REST consumidas por aplicaciones de página única (SPA) como el frontend en React.

Flujo de autenticación:

1. El usuario envía sus credenciales (correo electrónico y contraseña) al endpoint POST /api/user/login.

2. El AuthenticationManager de Spring Security valida las credenciales contra la base de datos, verificando que el correo existe y que la contraseña coincide con el hash BCrypt almacenado.

3. Si la autenticación es exitosa, el JwtService genera un token JWT firmado con el algoritmo HS512 que contiene el identificador del usuario (UUID), su rol (CUSTOMER o COMPANY) y las fechas de emisión y expiración (120 minutos por defecto).

4. El token se devuelve al cliente junto con los datos básicos del usuario (nombre, correo, rol e imagen de perfil).

5. En las peticiones posteriores, el cliente incluye el token en la cabecera HTTP Authorization con el esquema Bearer.

6. El JwtAuthenticationFilter intercepta cada petición entrante, extrae el token de la cabecera, valida su firma y expiración, carga los datos del usuario desde la base de datos y establece el contexto de seguridad de Spring.

7. Spring Security verifica que el usuario autenticado tiene el rol necesario para acceder al endpoint solicitado según las reglas definidas en SecurityConfig.

Configuración de acceso por roles:

La configuración de seguridad define tres niveles de acceso a los endpoints de la API:

• Endpoints públicos (sin autenticación): Incluyen el login, el registro de clientes y empresas, la recuperación de contraseña, todos los endpoints del GeneralController (catálogo de viajes, guías e imágenes), el formulario de contacto y la documentación Swagger.

• Endpoints de cliente (rol CUSTOMER): Todos los endpoints bajo /api/customer/ requieren que el usuario autenticado tenga el rol CUSTOMER. Incluyen la gestión de favoritos, el carrito de compras, el checkout y el historial de reservas.

• Endpoints de empresa (rol COMPANY): Todos los endpoints bajo /api/company/ requieren el rol COMPANY. Incluyen la creación y eliminación de guías y viajes.

Adicionalmente, se implementa una capa de verificación de propiedad en los servicios: cuando una empresa intenta eliminar un guía o un viaje, el sistema verifica que el recurso pertenece a la empresa autenticada, lanzando una excepción DifferentUserException en caso contrario.

Cifrado de contraseñas:

Todas las contraseñas se almacenan hasheadas mediante el algoritmo BCrypt, que incorpora un salt aleatorio en cada hash y un factor de coste configurable que dificulta los ataques de fuerza bruta. El mismo algoritmo se utiliza para hashear los códigos de recuperación de contraseña antes de almacenarlos en la base de datos.

Flujo de Recuperación de Contraseña

AventuraTrips implementa un flujo completo de recuperación de contraseña en tres pasos, diseñado con múltiples capas de seguridad:

1. Solicitud del código (POST /api/user/forgotPassword): El usuario proporciona su correo electrónico. El sistema genera un código numérico aleatorio de 6 dígitos, lo hashea con BCrypt antes de almacenarlo en la base de datos, establece una fecha de expiración de 10 minutos, reinicia el contador de intentos a cero y envía el código en texto plano al correo del usuario mediante Resend.

2. Verificación del código (POST /api/user/verifyResetCode): El usuario introduce el código recibido por correo. El sistema verifica que el código no ha expirado (ventana de 10 minutos), que no se han superado los 3 intentos máximos de verificación, y que el código proporcionado coincide con el hash almacenado. Si la verificación es exitosa, se activa un indicador (reset_code_verify) que autoriza el cambio de contraseña.

3. Establecimiento de la nueva contraseña (POST /api/user/resetPassword): El usuario proporciona su nueva contraseña. El sistema verifica que el indicador de verificación está activo, hashea la nueva contraseña con BCrypt, la almacena en la base de datos y limpia todos los campos de recuperación (código, expiración, intentos e indicador).

API REST: Endpoints y Controladores

La API REST de AventuraTrips se organiza en cuatro controladores que agrupan los endpoints según su dominio funcional. Todos los endpoints siguen el formato de respuesta estandarizado ApiResponse, que encapsula un código de estado, un mensaje descriptivo y los datos de la operación.

UserController (/api/user)

Este controlador gestiona la autenticación y la cuenta de usuario. Expone los endpoints de login que devuelve el token JWT y los datos del usuario, los tres endpoints del flujo de recuperación de contraseña (solicitud, verificación y reseteo), y el endpoint de eliminación de cuenta que propaga la eliminación en cascada a todas las entidades relacionadas del usuario, incluyendo la eliminación de su imagen de perfil de Cloudinary.

CompanyController (/api/company)

El controlador de empresa gestiona el registro de nuevas empresas y las operaciones CRUD sobre guías y viajes. El registro valida la unicidad de todos los campos corporativos (correo, nombre de empresa, CIF, teléfono e IBAN). La creación de guías acepta una imagen mediante multipart/form-data, la sube a Cloudinary y asocia el guía a la empresa autenticada. La creación de viajes soporta entre 1 y 5 imágenes simultáneas, genera automáticamente un rating aleatorio entre 4.0 y 5.0, y valida que las fechas de inicio y fin sean futuras y coherentes. Las operaciones de eliminación verifican la propiedad del recurso y eliminan las imágenes asociadas de Cloudinary.

CustomerController (/api/customer)

Este controlador implementa todas las funcionalidades del cliente. Permite gestionar la lista de favoritos (añadir, eliminar y consultar con paginación de 9 elementos por página), el carrito de compras (añadir, eliminar y consultar), el proceso de checkout y el historial de reservas con posibilidad de cancelación. Cada operación valida la autenticación del usuario y verifica que los viajes referenciados existen y no están duplicados en la colección correspondiente.

GeneralController (/api/general)

El controlador general expone los endpoints públicos accesibles sin autenticación. Incluye la obtención de viajes aleatorios (limitada a 10 por petición), imágenes aleatorias (limitada a 40 por petición), el listado completo de guías con su edad calculada dinámicamente, el catálogo paginado de viajes (9 por página, ordenados por fecha de inicio

ascendente), los viajes con descuento, el detalle completo de un viaje individual (incluyendo contadores simulados de espectadores y ventas), y el formulario de contacto que envía un correo electrónico con los datos del remitente. Opcionalmente, los endpoints de listado aceptan un parámetro email que, cuando se proporciona, excluye de los resultados los viajes que el cliente ya tiene en su carrito o en sus reservas.

Proceso de Checkout

El proceso de checkout es la operación transaccional más compleja del sistema y se ejecuta dentro de una única transacción de base de datos para garantizar la consistencia de los datos. Su implementación sigue los siguientes pasos:

1. Recepción de datos: El cliente envía un objeto CheckoutInDTO que contiene sus datos de facturación (nombre, correo, fecha de nacimiento, sexo, teléfono, número de documento y dirección postal completa) junto con la lista de identificadores de los viajes a reservar.

2. Bloqueo pesimista: Para cada viaje de la lista, el sistema ejecuta una consulta SELECT ... FOR UPDATE que bloquea el registro en la base de datos, impidiendo que otras transacciones concurrentes modifiquen el stock simultáneamente. Este mecanismo previene condiciones de carrera que podrían resultar en sobreventa de plazas.

3. Validación de duplicados: Se verifica que no exista una reserva previa con el mismo número de documento para cada viaje. Esta validación permite que diferentes personas realicen reservas desde la misma cuenta de cliente, pero impide que una misma persona (identificada por su documento) reserve el mismo viaje dos veces.

4. Verificación de stock: Se comprueba que cada viaje tenga al menos una plaza disponible. Si algún viaje no tiene stock suficiente, se lanza una excepción InsufficientStockException que incluye los identificadores de los viajes sin disponibilidad.

5. Creación de reservas: Para cada viaje validado se crea un registro en la tabla bookings con todos los datos de facturación, el precio del viaje en el momento de la compra (aplicando descuento si corresponde) y la marca temporal de la transacción.

6. Actualización de stock: Se decrementa en una unidad el stock de plazas disponibles de cada viaje reservado.

7. Limpieza de carrito y favoritos: Se eliminan los viajes comprados tanto del carrito como de la lista de favoritos del cliente, evitando referencias inconsistentes.

8. Envío de confirmación: Se genera un correo electrónico HTML con formato de recibo que incluye una tabla con el detalle de cada viaje reservado (título y precio) y el importe total pagado, y se envía al correo del cliente mediante Resend.

9. Respuesta: Se devuelve al cliente un objeto CheckoutOutDTO con la lista de títulos de los viajes reservados y el precio total formateado con el símbolo de la divisa correspondiente.

Manejo Global de Errores

La aplicación implementa un manejador global de excepciones mediante la anotación @RestControllerAdvice que intercepta todas las excepciones lanzadas en cualquier capa del sistema y las transforma en respuestas HTTP estandarizadas. Se han definido 21 excepciones personalizadas organizadas en las siguientes categorías:

• Errores de validación (HTTP 400 Bad Request): Incluyen datos duplicados en el registro, credenciales de login inválidas, formatos de imagen incorrectos, límites de API excedidos, carrito vacío y códigos de recuperación inválidos. También se capturan las excepciones de validación de Jakarta (MethodArgumentNotValidException) para devolver una lista detallada de los campos que no cumplen las restricciones.

• Recursos no encontrados (HTTP 404 Not Found): Se lanzan cuando se intenta acceder a un viaje, guía, usuario, favorito, elemento del carrito o reserva que no existe en la base de datos.

• Conflictos (HTTP 409 Conflict): Cubren los escenarios de elementos duplicados en favoritos o carrito, reservas duplicadas por documento de identidad y stock insuficiente durante el checkout.

• Errores de autorización (HTTP 401/403): Incluyen credenciales incorrectas (401 Unauthorized), acceso denegado por rol insuficiente (403 Forbidden) y operaciones sobre recursos que pertenecen a otro usuario (403 Forbidden).

• Errores de servidor (HTTP 500 Internal Server Error): Se producen cuando fallan las operaciones con servicios externos como la carga o eliminación de imágenes en Cloudinary o el envío de correos mediante Resend.

Todas las respuestas de error siguen el formato uniforme ApiResponse con tres campos: status (código de estado descriptivo), message (mensaje explicativo para el usuario) y data (información adicional sobre el error, como la lista de campos inválidos o los identificadores de los recursos afectados).

Integraciones con Servicios Externos

Integración con Cloudinary:

Cloudinary se utiliza como plataforma de almacenamiento y distribución de todas las imágenes de la aplicación. La integración se realiza mediante el SDK oficial cloudinary-

http44, configurado como un bean de Spring con las credenciales (cloud name, API key y API secret) inyectadas desde variables de entorno. El servicio CloudinaryService expone dos operaciones: upload, que recibe un archivo MultipartFile, lo sube a la carpeta tfg/ de Cloudinary y devuelve la URL segura HTTPS del recurso; y delete, que extrae el identificador público de la imagen a partir de su URL y solicita su eliminación del almacenamiento. Antes de cada carga, el sistema valida que el tipo MIME del archivo sea de tipo imagen. Las imágenes se sirven a través de la CDN global de Cloudinary, lo que garantiza tiempos de carga óptimos independientemente de la ubicación geográfica del usuario.

Integración con Resend:

Resend es el servicio de correo electrónico transaccional utilizado para todas las comunicaciones automatizadas de la plataforma. La configuración se realiza mediante un bean de Spring que instancia el cliente Resend con la API key proporcionada por variables de entorno. El servicio ResendService ofrece un método genérico sendEmail que acepta el destinatario, el asunto y el cuerpo HTML del mensaje. Los correos se envían desde la dirección configurada en la variable RESEND_DOMAIN. La aplicación utiliza este servicio en cuatro contextos: el envío de códigos de recuperación de contraseña, los recibos de compra tras el checkout (con tablas HTML detallando los viajes y el total), las confirmaciones de cancelación de reservas y las notificaciones del formulario de contacto dirigidas al correo corporativo de AventuraTrips.

Despliegue con Docker

El despliegue de la aplicación se gestiona mediante Docker y Docker Compose, lo que permite contenerizar tanto la aplicación Spring Boot como la base de datos MySQL en un entorno reproducible y portable. El archivo docker-compose.yml define dos servicios:

• Servicio MySQL (mysql:8.4): Ejecuta el motor de base de datos en un contenedor aislado con un volumen persistente (mysql-data) que garantiza la preservación de los datos entre reinicios del contenedor. Se configura con un health check basado en mysqladmin ping que verifica la disponibilidad del servicio antes de permitir que la aplicación se conecte.

• Servicio de la aplicación Spring Boot: Se construye a partir del Dockerfile del proyecto, expone el puerto 8080 y se configura para reiniciarse automáticamente en caso de fallo (restart: always). La conexión con la base de datos y las credenciales de los servicios externos se inyectan mediante variables de entorno, siguiendo la metodología de los doce factores (twelve-factor app) para aplicaciones nativas de la nube.

La dependencia entre servicios se gestiona mediante la directiva depends_on con la condición service_healthy, lo que asegura que el contenedor de la aplicación no se inicie hasta que MySQL esté completamente operativo y respondiendo a conexiones. Este mecanismo evita errores de conexión durante el arranque inicial del sistema.

Documentación de la API con Swagger

La API REST de AventuraTrips se documenta de forma interactiva mediante SpringDoc OpenAPI, que genera automáticamente una interfaz Swagger UI accesible en la ruta /swagger-ui.html. Esta documentación permite a los desarrolladores y evaluadores explorar todos los endpoints disponibles, visualizar los esquemas de los DTOs de petición y respuesta, probar las operaciones directamente desde el navegador y autenticarse mediante el esquema Bearer JWT. La configuración de OpenAPI incluye la definición del esquema de seguridad que permite introducir el token JWT en la interfaz de Swagger para probar los endpoints protegidos sin necesidad de utilizar herramientas externas como Postman.

Trabajos futuros

Tras la culminación de la fase actual de AventuraTrips, se han identificado diversas líneas de expansión y mejora que permitirían escalar el proyecto hacia una plataforma comercialmente más competitiva y funcional. Los desarrollos futuros se centran en cuatro ejes principales:

1. Portal de Gestión para Empresas (B2B Dashboard)

Aunque actualmente el sistema ya soporta el rol de Company a nivel de API y base de datos, la gestión de viajes y guías se realiza de forma técnica. El objetivo es evolucionar hacia una plataforma de autoservicio mediante un Dashboard especializado que permita a las empresas::

- Autogestión de inventario: Interfaz intuitiva para que las empresas creen, editen y retiren sus paquetes turísticos en tiempo real.
- Analítica de ventas: Visualización de métricas sobre los destinos más reservados y el volumen de facturación por período.
- Gestión de Guías: Herramientas de asignación dinámica de guías a viajes según disponibilidad y especialidad.

2. Escalabilidad del Catálogo y Nuevas Verticales

Para posicionar a AventuraTrips como un referente en el sector, se planea una expansión masiva del catálogo bajo dos estrategias:

- Aumento de oferta: Incremento de la base de datos de destinos y paquetes turísticos cubriendo los cinco continentes.
- Nuevas líneas de producto: Integración de una vertical de Excursiones de un día y actividades locales, permitiendo a los usuarios complementar sus viajes principales con experiencias puntuales.

3. Optimización de la Arquitectura de Búsqueda

Con el fin de mejorar la conversión y la experiencia de usuario (UX), se proyecta la implementación de un Motor de Búsqueda Avanzado en la página de inicio que incluya:

- Filtrado Multicriterio: Capacidad de búsqueda por nombre del destino, país, rango de precios y categorías.
- Selector de Fechas Dinámico: Filtro por calendario para mostrar únicamente los paquetes con disponibilidad en las fechas seleccionadas por el usuario, integrando validaciones en tiempo real con el stock del backend.

4. Sistema de Feedback y Prueba Social (Social Proof)

Para aumentar la confianza de los nuevos clientes, se activará el modelo de Comentarios y Valoraciones (entidad comments ya definida en el esquema de base de datos):

- Reseñas Verificadas: Permitir que solo los usuarios que han completado un viaje (estatus "comprado") puedan dejar una valoración numérica y un comentario textual.
- Display de Producto: Integración de una sección de reseñas en la vista de detalle del viaje, calculando la media de valoraciones de forma dinámica para mostrarla en los *cards* del catálogo.

Conclusiones

La realización del proyecto AventuraTrips ha supuesto un reto integral que ha permitido consolidar los conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Web (DAW), transformando conceptos teóricos en una solución digital funcional, escalable y con un enfoque comercial realista.

A continuación, se detallan las principales conclusiones extraídas tras la finalización del desarrollo:

1. Consolidación de la Arquitectura Full-Stack

El desarrollo de una aplicación con un desacoplamiento total entre el Frontend (React) y el Backend (Spring Boot) ha demostrado ser una estrategia eficaz para garantizar la mantenibilidad del sistema. La implementación de una API REST robusta, documentada con Swagger y protegida mediante JWT, asegura que la plataforma pueda crecer o integrarse con otros servicios en el futuro sin comprometer la seguridad ni el rendimiento.

2. Seguridad y Gestión de Datos

Uno de los mayores aprendizajes ha sido la gestión de la persistencia y la seguridad. El uso de Spring Security junto con técnicas avanzadas como el bloqueo pesimista (pessimistic locking) en la base de datos MySQL, ha permitido resolver problemas críticos de concurrencia, como la sobreventa de plazas en los viajes (stock), garantizando la integridad de las transacciones financieras y de reserva.

3. Innovación mediante Inteligencia Artificial

La integración de IA Generativa para la creación de activos visuales ha marcado una diferencia competitiva en el proyecto. Esta tecnología nos ha permitido dotar a AventuraTrips de una identidad visual única y coherente, eliminando la dependencia de bancos de imágenes genéricos y demostrando cómo la IA puede ser una herramienta aliada en el flujo de trabajo de un desarrollador moderno.

4. Experiencia de Usuario y Valor Añadido

Más allá de la funcionalidad básica de compra, el proyecto ha hecho hincapié en la UX (User Experience) mediante detalles como el chatbot de asistencia personalizada, el diseño adaptativo (responsive) y la gestión eficiente de sesiones. Estas características elevan el proyecto de un simple catálogo a una plataforma de e-commerce real que prioriza la comodidad y la retención del usuario.

5. Gestión del Ciclo de Vida del Software

El uso de herramientas como Docker para la contenerización y Git para el control de versiones ha sido fundamental para entender la importancia de los entornos de despliegue reproducibles. La planificación mediante diagramas de Gantt y el seguimiento de requisitos

(RFTP) nos han permitido cumplir con los plazos de entrega y gestionar las desviaciones de manera profesional.

Valoración Final

En conclusión, AventuraTrips no es solo una plataforma de venta de viajes, sino el resultado de un proceso de ingeniería donde se han cuidado todas las fases: desde el diseño de la base de datos hasta la interfaz de usuario final. El proyecto cumple con todos los objetivos propuestos inicialmente y sienta las bases para una futura expansión hacia un modelo de negocio B2B más complejo.

Referencias

Anthropic. (2026). Claude (versión 4.6) [Modelo de lenguaje grande]. https://claude.ai/

Broadcom. (2024). *Spring Framework reference documentation*. https://docs.spring.io/spring-framework/reference/index.html

Meta Platforms. (2024). *React: The library for web and native user interfaces*. https://react.dev/

Reed, A. (2023). Desarrollo Java completo con Spring Boot y React. Independently published.

Tascón Gómez, M., & Longarte González, J. (2026). AventuraTrips (versión 1.0) [Software]. GitHub. https://github.com/Jlongarte/AventuraT